

Spendly / Financial Tracker Mobile App — Detailed Technical Demo Preparation Guide

Spendly / Financial Tracker Mobile App — Detailed Technical Demo Preparation Guide

Prepared for a 15-minute live demo and engineering viva.

Repository inspected: /Users/chamikalakshan/Documents/Codex/Financial-Tracker-Mobile-Kotlin

Generated date: 2026-06-05

1. Project Overview

1.1 What The Application Does

Spendly is a Kotlin Android mobile application for personal finance tracking. It helps a user create an account, sign in, record income, record expenses, track savings goals, review recent transaction history, view monthly analytics, and manage profile preferences such as default currency and app appearance.

The app is built around a local-first data flow. User-facing screens read data from local Room database streams through ViewModels, while repositories synchronize local records with Firebase Firestore. This means the app can continue to show local cached information and can keep unsynced records until a network sync succeeds.

1.2 Main Problem Solved

The core problem is that personal finance data is often scattered or manually calculated. Spendly centralizes the user's financial records and converts them into useful summaries:

Problem	Spendly Solution
Users forget where money goes	Expenses are categorized and listed by month
Users cannot quickly see savings progress	Dashboard shows net savings and goal progress
Users need account-based persistence	Firebase Authentication identifies each user
Users may use multiple devices	Firestore stores user-owned cloud data
Users may be temporarily offline	Room caches records locally and sync runs later
Users need quick insights	Analytics aggregates income, expenses, categories, and monthly trends

1.3 Target Users

The target users are individuals who want a lightweight mobile finance tracker. The application is especially suitable for students, freelancers, and young professionals who want to track income sources, expenses, and savings goals without using a complex accounting system.

1.4 Core Features

Feature	What It Allows The User To Do	Main Files
Authentication	Sign in, create account, reset password, change password	AuthScreen.kt, CreateAccountScreen.kt, FirebaseAuthRepository.kt
Dashboard	View balance, income, expense, net savings, goals, recent transactions	HomeScreen.kt, FinanceViewModel.kt
Transactions / History	Add, edit, delete, filter, and group income/expense records	TransactionsScreen.kt, AddIncomeScreen.kt, AddExpenseScreen.kt
Goals	Create goals, edit goals, add savings, track progress	GoalsScreen.kt, GoalsViewModel.kt, GoalRepositoryImpl.kt
Analytics	View monthly totals, category breakdown, spending split, 5-month overview	AnalyticsScreen.kt, AnalyticsViewModel.kt
Profile	Edit name/photo, currency, appearance, exchange rate, password, logout	ProfileScreen.kt, FinanceViewModel.kt, UserRepositoryImpl.kt
Sync	Upload/download profile, income, expenses, and goals	SpendlySyncWorker.kt, SyncManager.kt

1.5 Why The Project Is Useful

Spendly demonstrates a real mobile application architecture rather than only static UI screens. It includes authentication, local database caching, remote cloud persistence, dependency injection, background work, navigation, theme management, and screen-level state management.

For a coursework viva, this is useful because the project can be explained at both user-flow level and engineering level:

- User-flow level: “A user signs in, adds income, records expenses, tracks a savings goal, and views analytics.”
- Engineering level: “A Compose screen calls a ViewModel. The ViewModel calls a repository. The repository writes to Room first and then syncs to Firestore. WorkManager later retries unsynced records.”

1.6 Coursework Requirement Alignment

The implementation aligns with common mobile development coursework requirements:

Requirement Area	Codebase Evidence
Native Android development	Kotlin Android app with Jetpack Compose
Modern UI framework	Compose Material 3 components
MVVM architecture	ui/viewmodel, repositories, model classes
Local database	Room entities, DAOs, and SpendlyDatabase.kt
Cloud backend	Firebase Auth and Firestore repository implementations
Dependency injection	Hilt modules and injected ViewModels
Background processing	WorkManager sync worker
Navigation	Navigation Compose in FinanceTrackerApp.kt and ui/navigation
Team domain separation	Dashboard/Profile, Transactions/Register, Goals/Login, Analytics/DB/Sync

1.7 Short Demo Opening Script

“Spendly is a personal finance tracker built with Kotlin, Jetpack Compose, MVVM, Room, Firebase Firestore, Hilt, and WorkManager. The app lets a user create an account, record income and expenses, track savings goals, and view monthly analytics. Technically, the app follows a local-first architecture: records are saved to Room first, then synced to Firestore. ViewModels expose StateFlow to Compose screens, and Hilt manages dependencies across repositories, DAOs, Firebase, and WorkManager.”

2. Technology Stack

2.1 Kotlin

Kotlin is the main programming language of the app.

Why Used	Where Used	Viva Explanation
Modern Android language with null safety and coroutine support	All .kt source files under app/src/main/kotlin/com/spendly/financetracker	“Kotlin gives concise syntax, null safety, data classes, sealed classes, coroutines, and works naturally with Compose and Room.”

Important examples:

- data/model/FinanceTransaction.kt uses Kotlin data classes and enums for transaction models.
- ui/viewmodel/FinanceViewModel.kt uses coroutines and StateFlow.
- data/remote/*RepositoryImpl.kt uses suspend functions and coroutine `await()` calls for Firebase tasks.

2.2 Jetpack Compose

Jetpack Compose is used for UI.

Why Used	Where Used	Viva Explanation
Declarative UI, state-driven rendering, Material 3 integration	ui/screen, ui/components, ui/theme	“Compose screens observe ViewModel state and redraw automatically when StateFlow changes.”

Important files:

- ui/FinanceTrackerApp.kt
- ui/screen/home/HomeScreen.kt
- ui/screen/transactions/TransactionsScreen.kt
- ui/screen/analytics/AnalyticsScreen.kt
- ui/screen/goals/GoalsScreen.kt
- ui/screen/profile/ProfileScreen.kt

2.3 MVVM Architecture

The app follows Model-View-ViewModel.

MVVM Part	Codebase Location	Responsibility
Model	data/model, data/local/entity	Defines transaction, goal, user, and session data
View	ui/screen, ui/components	Displays UI and sends user events
ViewModel	ui/viewmodel	Holds screen state, validates inputs, calls repositories
Repository	data/repository, data/remote	Abstracts Room and Firestore access

Viva explanation:

“The Compose UI does not directly call Room or Firestore. It sends events to a ViewModel. The ViewModel updates state and calls repository interfaces. Repository implementations decide whether to read/write Room, Firestore, or both.”

2.4 Firebase Authentication

Firebase Authentication handles user accounts.

Why Used	Where Used
Provides secure account creation, login, logout, reset password, update password	FirebaseAuthRepository.kt, AppModule.kt, FinanceViewModel.kt, CreateAccountViewModel.kt

Important operations:

- `createAccount(name, email, password, defaultCurrency)`
- `signIn(email, password)`
- `signOut()`
- `sendPasswordResetEmail(email)`
- `updatePassword(currentPassword, newPassword)`
- `observeSession()`

2.5 Firebase Firestore

Firestore is the remote cloud database.

Why Used	Where Used
Stores profile, income, expenses, and goals per Firebase user	data/remote, FirebaseAuthRepository.kt, AppModule.kt

Collections used in code:

```

users/{uid}/profile/main
users/{uid}/income/{incomeId}
users/{uid}/expenses/{expenseId}
users/{uid}/goals/{goalId}

```

Viva explanation:

“The Firebase UID is the owner key. Each user’s data is stored under `users/{uid}`. That prevents mixing data between users and supports cross-device sync.”

2.6 Room Database

Room provides local persistence.

Why Used	Where Used
Offline cache, local query streams, fast UI updates	<code>data/local/entity</code> , <code>data/local/dao</code> , <code>SpendlyDatabase.kt</code>

Important entities:

- `IncomeEntryEntity`
- `ExpenseEntryEntity`
- `SavingsGoalEntity`
- `UserProfileEntity`

Important DAOs:

- `IncomeDao`
- `ExpenseDao`
- `GoalDao`
- `UserProfileDao`

2.7 Hilt Dependency Injection

Hilt creates and injects dependencies.

Why Used	Where Used
Avoids manual object creation and supports testable architecture	<code>SpendlyApplication.kt</code> , <code>MainActivity.kt</code> , <code>di/AppModule.kt</code> , <code>di/RepositoryModule.kt</code> , <code>ViewModels</code> , <code>worker</code>

Important annotations:

- `@HiltAndroidApp` in `SpendlyApplication.kt`
- `@AndroidEntryPoint` in `MainActivity.kt`
- `@Module`, `@InstallIn`, `@Provides`, `@Binds`
- `@HiltViewModel`
- `@HiltWorker`

2.8 WorkManager

WorkManager handles background sync.

Why Used	Where Used
Runs reliable sync work with network constraints	worker/SpendlySyncWorker.kt, util/SyncManager.kt, SpendlyApplication.kt

Viva explanation:

“When writes fail or records remain unsynced, WorkManager can retry later when the device has a network connection.”

2.9 Coroutines, Flow, And StateFlow

The code uses coroutines and flows for asynchronous work and observable UI state.

API	Example Usage
<code>viewModelScope.launch</code>	ViewModels run repository calls without blocking UI
Flow	DAOs expose observable database streams
StateFlow	ViewModels expose immutable UI state to Compose
<code>combine</code>	FinanceViewModel combines profile, transactions, and goals

2.10 Navigation Compose

Navigation Compose controls screen routing.

Important files:

- `ui/FinanceTrackerApp.kt`
- `ui/navigation/Screen.kt`
- `ui/navigation/BottomNavItem.kt`
- `ui/navigation/SpendlyNavGraph.kt`
- `ui/navigation/SpendlyBottomNavBar.kt`

The active implementation is mainly inside `FinanceTrackerApp.kt`, which builds the scaffold, bottom navigation, `NavHost`, routes, transitions, and shared floating action button.

2.11 Material Design 3

Material 3 is used for modern Compose UI components.

Examples:

- Scaffold
- NavigationBar
- NavigationBarItem
- TopAppBar
- Card
- Button
- OutlinedTextField
- AlertDialog
- FloatingActionButton
- FilterChip
- ModalBottomSheet

2.12 Exchange Rate And Crypto API Services

The codebase includes simple service classes:

- `data/service/CurrencyRateService.kt`
- `data/service/CryptoRateService.kt`

Currency conversion is optional. If live rates are unavailable, the UI supports manual rate entry. This is important for demo explanation because saving income/expense does not depend completely on external API availability.

2.13 Gradle Dependencies

The main dependency file is:

`app/build.gradle.kts`

Important configured items:

Dependency / Plugin	Purpose
Android application plugin	Build Android app
Kotlin Android plugin	Kotlin support
Google services plugin	Firebase integration
Hilt plugin	Dependency injection
KSP plugin	Room/Hilt annotation processing
Compose BOM and Material 3	UI
Navigation Compose	Routing
Room runtime/ktx/compiler	Local database
Firebase BOM, Auth, Firestore	Backend
WorkManager and Hilt Work	Sync
Coroutines Play Services	Firebase <code>await()</code> support

3. Full Folder And File Structure Explanation

3.1 High-Level Project Tree

```

Financial-Tracker-Mobile-Kotlin/
├── MainActivity.kt
├── SpendlyApplication.kt
├── app/
│   ├── build.gradle.kts
│   ├── google-services.json
│   └── src/
│       ├── main/
│       │   ├── AndroidManifest.xml
│       │   └── kotlin/com/spendly/financetracker/
│       │       ├── data/
│       │       │   ├── firebase/
│       │       │   ├── local/
│       │       │   │   ├── dao/
│       │       │   │   ├── db/
│       │       │   │   └── entity/
│       │       │   ├── model/
│       │       │   ├── remote/
│       │       │   ├── repository/
│       │       │   └── service/
│       │       ├── di/
│       │       ├── ui/
│       │       │   ├── components/
│       │       │   ├── navigation/
│       │       │   ├── screen/
│       │       │   │   ├── analytics/
│       │       │   │   ├── goals/
│       │       │   │   ├── home/
│       │       │   │   ├── profile/
│       │       │   │   └── transactions/
│       │       │   ├── theme/
│       │       │   ├── util/
│       │       │   └── viewmodel/
│       │       ├── util/
│       │       └── worker/
│       └── res/
├── docs/
├── firestore.indexes.json
├── firestore.rules
├── build.gradle.kts
├── settings.gradle.kts
└── gradle/libs.versions.toml
  
```

3.2 Folder Responsibilities

Folder	Layer	Responsibility
data/model	Domain model	App-level data classes used by UI and ViewModels
data/local/entity	Room schema	Database table definitions
data/local/dao	Room access	SQL queries and observable flows
data/local/db	Room database	Database class and migrations
data/repository	Repository contracts	Interfaces for clean architecture
data/remote	Repository implementations	Room + Firestore integration
data/service	External services	Currency and crypto rate helpers
data/firebase	Firestore setup helpers	Firestore bootstrap checks
di	Dependency injection	Hilt modules
ui/screen	UI screens	Compose page-level screens
ui/components	Shared UI	Reusable cards, nav, rows, controls
ui/navigation	Navigation definitions	Route names and bottom nav data
ui/theme	Theme system	Colors, typography, light/dark modes
ui/util	UI helpers	Money formatting, month options, category settings
ui/viewmodel	ViewModel layer	State and business logic for screens
util	App utilities	Mapping and sync orchestration
worker	Background work	WorkManager sync worker

3.3 Important File Table

File Path	Layer	Main
MainActivity.kt	App entry	Starts C applies l
SpendlyApplication.kt	App entry / DI	Enables schedul

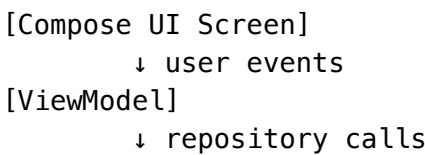
File Path	Layer	Main
ui/FinanceTrackerApp.kt	UI shell	App rou bottom 1
ui/navigation/Screen.kt	Navigation	Route c
ui/navigation/BottomNavItem.kt	Navigation	Bottom
ui/components/SpendlyAddActionMenu.kt	UI component	Shared j
ui/components/SpendlyMonthPicker.kt	UI component	Shared 1
ui/components/TransactionListItem.kt	UI component	Transac
ui/screen/home/HomeScreen.kt	UI screen	Dashbo: summar
ui/screen/profile/ProfileScreen.kt	UI screen	Profile/s
ui/screen/transactions/TransactionsScreen.kt	UI screen	History list/filte
ui/screen/transactions/AddIncomeScreen.kt	UI screen	Add/edi
ui/screen/transactions/AddExpenseScreen.kt	UI screen	Add/edi
ui/screen/analytics/AnalyticsScreen.kt	UI screen	Analytic cards
ui/screen/goals/GoalsScreen.kt	UI screen	Goal lis
ui/screen/AuthScreen.kt	UI screen	Login U
ui/screen/CreateAccountScreen.kt	UI screen	Registra
ui/viewmodel/FinanceViewModel.kt	ViewModel	Global s state
ui/viewmodel/TransactionsViewModel.kt	ViewModel	History delete
ui/viewmodel/AddIncomeViewModel.kt	ViewModel	Income validatio
ui/viewmodel/AddExpenseViewModel.kt	ViewModel	Expense validatio
ui/viewmodel/AnalyticsViewModel.kt	ViewModel	Analytic
ui/viewmodel/GoalsViewModel.kt	ViewModel	Goal form/de
ui/viewmodel/CreateAccountViewModel.kt	ViewModel	Registra
data/local/db/SpendlyDatabase.kt	Room	Databas
data/local/dao/IncomeDao.kt	DAO	Income
data/local/dao/ExpenseDao.kt	DAO	Expense operatio

File Path	Layer	Main
data/local/dao/GoalDao.kt	DAO	Goal SC
data/local/dao/UserProfileDao.kt	DAO	Profile S
data/remote/IncomeRepositoryImpl.kt	Repository impl	Room + income
data/remote/ExpenseRepositoryImpl.kt	Repository impl	Room + expense
data/remote/GoalRepositoryImpl.kt	Repository impl	Goal pe savings
data/remote/UserRepositoryImpl.kt	Repository impl	Profile f sync
data/repository/FirebaseAuthRepository.kt	Repository impl	Firebase integrati
data/repository/FirebaseTransactionRepository.kt	Repository impl	Combin expense
di/AppModule.kt	DI	Provide: Room, I WorkM:
di/RepositoryModule.kt	DI	Binds re interfac
worker/SpendlySyncWorker.kt	WorkManager	Backgrc worker
util/SyncManager.kt	Utility	Schedul
util/Mappers.kt	Mapping	Convert entities/ maps
ui/theme/Theme.kt	Theme	Material theme
ui/theme/ThemeMode.kt	Theme	Theme 1
ui/util/UiUtils.kt	UI utility	Amount

4. Architecture Explanation

4.1 Architecture Style

The app uses a layered MVVM architecture:



```

[Repository Interface]
    ↓ implementation
[Room DAO] + [Firebase Firestore]
    ↓
[Flow / suspend result]
    ↓
[ViewModel StateFlow]
    ↓
[Compose recomposition]

```

4.2 Main Data Flow

Example: user adds income.

```

User taps Save on AddIncomeScreen
    ↓
AddIncomeViewModel validates amount, source, date, currency
    ↓
IncomeRepository.addIncome(draft)
    ↓
IncomeRepositoryImpl inserts IncomeEntryEntity into Room with
isSynced=false
    ↓
IncomeRepositoryImpl tries to write users/{uid}/income/{id} in
Firestore
    ↓
If success, DAO marks local row as synced
    ↓
Room Flow emits updated income list
    ↓
FinanceViewModel / TransactionsViewModel receive new state
    ↓
Home and History screens update automatically

```

4.3 UI Layer

The UI layer is Compose-based. It should mainly:

- Display state.
- Collect state from ViewModels.
- Call callbacks when users interact.
- Avoid direct repository or Firebase access.

Main UI folders:

- ui/screen
- ui/components
- ui/theme
- ui/util

4.4 ViewModel Layer

ViewModels hold UI state and business rules. Examples:

ViewModel	Responsibility
FinanceViewModel	Global session, profile, dashboard state, sync trigger
TransactionsViewModel	Month filtering, transaction grouping, delete actions
AddIncomeViewModel	Income validation, currency conversion, source settings
AddExpenseViewModel	Expense validation, category/payment/type handling
AnalyticsViewModel	Monthly totals, category percentages, spending split
GoalsViewModel	Goal form state, progress, saving validation
CreateAccountViewModel	Register validation and account creation

4.5 Repository Layer

Repository interfaces live in `data/repository`. Implementations live in `data/remote`.

This split is important because ViewModels depend on interfaces instead of concrete Firebase/Room code. That improves testability and keeps UI code clean.

4.6 Local Database Layer

Room is used for:

- Local caching.
- Fast list rendering.
- Offline-created records.
- Observable Flow streams.

Each entity has an `isSynced` field. Repositories set `isSynced=false` when data is written locally. After Firestore upload succeeds, the corresponding DAO method marks the row as synced.

4.7 Remote Firestore Layer

Firestore stores cloud copies under the Firebase UID. The repository implementations read and write Firestore documents. The remote documents are mapped through `util/Mappers.kt`.

4.8 Dependency Injection Layer

Hilt provides:

- Firebase Auth.
- Firebase Firestore.
- Room database.
- DAOs.
- Repository implementations.
- WorkManager.
- Worker factory.

This avoids manually creating these dependencies inside UI screens.

4.9 Background Sync Layer

`SpendlySyncWorker` syncs profile, income, expenses, and goals. `SyncManager` schedules periodic work and immediate sync. `SpendlyApplication` schedules periodic sync when the app starts.

4.10 Navigation Layer

The main navigation shell is in `FinanceTrackerApp.kt`. It has:

- Splash route.
- Auth routes.
- Main app routes.
- Bottom navigation visibility rules.
- Shared FAB overlay.
- Slide/fade transitions.

4.11 Why This Architecture Is Suitable

This architecture is suitable because the app has real data, multiple screens, offline caching, background sync, and user accounts. A simple screen-only app would become difficult to maintain. MVVM separates responsibilities:

- UI remains focused on rendering.
- ViewModel handles state and validation.
- Repository handles data source decisions.
- Room handles local database.
- Firestore handles cloud persistence.
- WorkManager handles delayed sync.

5. Domain Breakdown By Team Member

5.1 Domain Allocation Summary

Member	Domain	Main Responsibility
Chamika	Dashboard and Profile	Dashboard UI/state, profile UI/settings, related profile/backend integration

Member	Domain	Main Responsibility
Yesen	Transactions and Create Account	Add/list/edit/delete income/expense, registration flow, transaction backend integration
Nikini	Goals and Login	Goal tracking, goal savings, login flow, related backend integration
Mahima	Analytics and DB/Sync foundation	Analytics, Room/Firestore setup, entities, caching, sync, WorkManager

5.2 Shared / Common Files

Some files are shared across all domains and should be explained as common architecture:

Shared File	Why Shared
MainActivity.kt	Entry point and theme setup
SpendlyApplication.kt	Hilt app and WorkManager setup
ui/FinanceTrackerApp.kt	Main navigation and scaffold
di/AppModule.kt	Shared dependency providers
di/RepositoryModule.kt	Repository binding
data/local/db/SpendlyDatabase.kt	Shared local DB
util/Mappers.kt	Shared mapping logic
util/SyncManager.kt	Shared sync orchestration
ui/util/UiUtils.kt	Shared formatting
ui/theme/*	Shared visual system

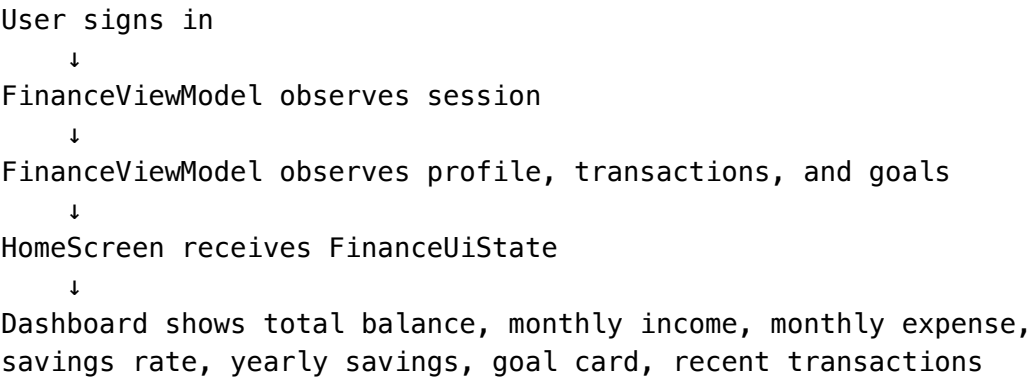
6. Chamika Domain — Profile Page And Dashboard Page

6.1 Feature Overview

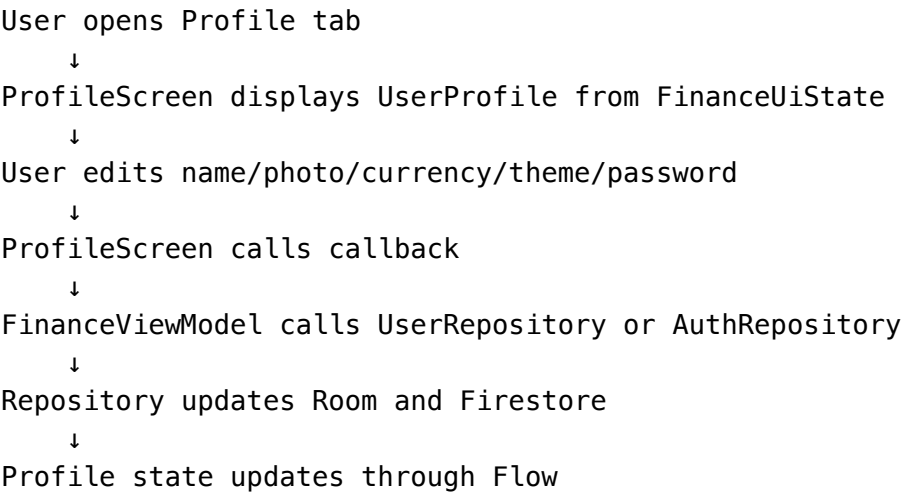
Chamika's domain covers:

- Dashboard page.
- Profile page.
- Dashboard/profile UI components.
- Dashboard summary calculations through app state.
- Profile update and settings flow.
- Profile data communication with Room and Firestore.

6.2 User Flow: Dashboard



6.3 User Flow: Profile



6.4 Related Files

File	Purpose
ui/screen/home/HomeScreen.kt	Dashboard UI
ui/screen/profile/ProfileScreen.kt	Profile/settings UI
ui/viewmodel/FinanceViewModel.kt	Global state and dashboard calculations
ui/viewmodel/FinanceUiState.kt	Dashboard/profile state model
ui/viewmodel/HomeViewModel.kt	Dashboard-specific ViewModel file, currently less central than FinanceViewModel
ui/viewmodel/ProfileViewModel.kt	Profile-specific ViewModel file, currently less central than FinanceViewModel
data/model/UserProfile.kt	Profile domain model
data/local/entity/UserProfileEntity.kt	Room profile entity
data/local/dao/UserProfileDao.kt	Room profile DAO
data/repository/UserRepository.kt	Profile repository contract

File	Purpose
data/remote/UserRepositoryImpl.kt	Room + Firestore profile implementation
ui/components/ProfileStat.kt	Profile stat component
ui/components/SummaryCard.kt	Dashboard summary card
ui/components/SummaryPanel.kt	Dashboard panel component
ui/components/HeaderSection.kt	Header component
ui/components/GoalCard.kt	Goal summary card
ui/components/TransactionListItem.kt	Recent transaction rows

6.5 Dashboard UI File Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/ui/screen/home/HomeScreen.kt

Purpose

This file builds the Dashboard screen. It displays:

- Green header.
- Greeting and profile avatar.
- Total balance.
- Monthly income card.
- Monthly expense card.
- Monthly net savings section.
- Yearly net savings text.
- Primary goal card.
- Recent transaction section.

Important Composable

HomeScreen(...)

Step-By-Step Logic

1. The screen receives `FinanceUiState`.
2. It reads user name/profile image/default currency from state.
3. It uses amount formatting helpers to display money with comma separators.
4. It displays current-month income and expense values.
5. It displays net savings and savings rate.
6. It displays yearly net savings.
7. It displays a primary goal if available.
8. It displays recent transactions if available.
9. Profile avatar click navigates to Profile.
10. Add actions are handled by the shared FAB in `FinanceTrackerApp.kt`, not directly inside this screen.

How It Connects To Backend

HomeScreen itself does not call the database. Data comes through `FinanceUiState`, which is built by `FinanceViewModel`. `FinanceViewModel` observes:

- `UserRepository.observeProfile(uid)`
- `TransactionRepository.observeTransactions(uid)`
- `GoalRepository.observeGoals(uid)`

Viva Questions

Question	Short Answer
Does Dashboard directly query Firestore?	No. Dashboard reads <code>FinanceUiState</code> ; repository/database access is handled by <code>ViewModel</code> and repositories.
Where is monthly income calculated?	In the state preparation logic, mainly <code>FinanceViewModel</code> , using transactions filtered by month/date.
Why use <code>dateMillis</code> ?	It represents the user-selected transaction date and is used for monthly grouping/display.
Why not use <code>createdAtMillis</code> for dashboard?	Created date is audit metadata. Financial reports should use the event date selected by the user.

6.6 Profile UI File Explanation

File Path

`app/src/main/kotlin/com/spendly/financetracker/ui/screen/profile/ProfileScreen.kt`

Purpose

This file builds the Profile page and settings UI.

Important UI Features

- Avatar or profile image display.
- Edit icon overlay near avatar.
- Name and email display.
- Edit profile dialog.
- Default currency dialog.
- Appearance selector: System, Light, Dark.
- Exchange rate settings dialog.
- Change password dialog.
- Logout confirmation.
- Delete account confirmation.

Step-By-Step Logic

1. The screen receives current profile/session state.
2. If a profile image URI exists, the screen attempts to load it.
3. If the image is missing or fails, initials are displayed.
4. User taps edit icon to change profile information.
5. User taps Default Currency to open currency selection.
6. User taps Appearance to change theme mode.
7. User taps Change Password to open password fields.
8. User taps Logout and confirms.
9. UI callbacks are sent upward to `FinanceTrackerApp.kt`, then to `FinanceViewModel`.

Profile Image Logic

The code uses a local URI approach. Profile image upload to Firebase Storage is not implemented.

Status:

Firebase Storage profile image upload: Not found in current codebase.

Profile Backend Flow

```

ProfileScreen user event
    ↓
FinanceTrackerApp callback
    ↓
FinanceViewModel.updateProfile(...)
    ↓
UserRepository.upsertProfile(...)
    ↓
UserRepositoryImpl writes UserProfileEntity to Room
    ↓
UserRepositoryImpl writes users/{uid}/profile/main to Firestore
    ↓
UserProfileDao.observeById emits updated profile
  
```

6.7 Profile Repository File Explanation

File Path

`app/src/main/kotlin/com/spendly/financetracker/data/remote/UserRepositoryImpl.kt`

Purpose

Handles profile read/write/sync between Room and Firestore.

Important Functions

Function	Purpose
<code>observeProfile(uid)</code>	Emits local profile from Room
<code>getProfile(uid)</code>	Reads profile once
<code>upsertProfile(profile)</code>	Saves profile locally and remotely
<code>syncWithFirestore(uid)</code>	Merges local unsynced and remote profile

Step-By-Step Profile Save

1. Convert `UserProfile` to `UserProfileEntity`.
2. Save entity to Room with `isSynced=false`.
3. Write expanded profile map to Firestore path `users/{uid}/profile/main`.
4. If Firestore write succeeds, mark profile as synced in Room.
5. If Firestore write fails, keep local data unsynced for later sync.

6.8 Dashboard Calculation Logic

Dashboard values are not hardcoded. They come from transaction and goal data:

Dashboard Value	Source
Total balance	Income minus expenses
Current-month income	Income transactions where <code>dateMillis</code> falls inside selected/current month
Current-month expense	Expense transactions where <code>dateMillis</code> falls inside selected/current month
Net savings	Current-month income minus current-month expenses
Savings rate	Net savings divided by income
Yearly net savings	Current calendar-year income minus expenses
Recent transactions	Latest transactions sorted by <code>dateMillis</code>
Primary goal	Goal marked as primary

6.9 Validation And Error Handling

Profile-related error handling exists through `ViewModel` state and dialog feedback. Change password may return Firebase reauthentication errors. The app supports success/error messages in state.

Dashboard mostly displays empty states when data is missing. Since Dashboard is read-only, validation is less relevant there.

6.10 Chamika Viva Questions And Answers

Question	Answer
How is the dashboard updated after adding a transaction?	The transaction is saved to Room. Room Flow emits a new list. <code>FinanceViewModel</code> recomputes state and <code>Compose</code> redraws Dashboard.
Does Profile update Firestore directly from the UI?	No. UI calls <code>ViewModel</code> callback. The repository writes Room and Firestore.
Where is the theme mode stored?	In <code>UserProfile.themeMode</code> and <code>UserProfileEntity.themeMode</code> ; it syncs through Firestore profile document.
What happens if Firestore profile update fails?	Local Room data remains with <code>isSynced=false</code> , and <code>WorkManager</code> can retry later.
Is profile image cloud-synced?	No. Current implementation stores local URI/path only. Firebase Storage is not included.

6.11 Demo Script For Chamika Domain

1. Open Dashboard.
2. Say: "This screen is data-driven from Room through `FinanceViewModel`."
3. Point to income, expenses, net savings, and recent transactions.
4. Add a transaction and return to Dashboard.
5. Show updated values.
6. Open Profile.
7. Change theme or currency.
8. Explain profile is stored in Room first and synced to Firestore profile document.

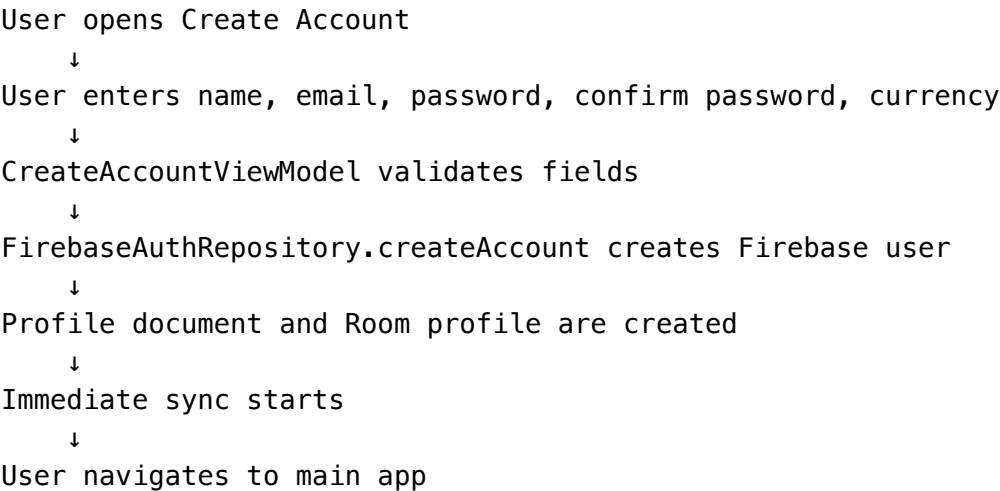
7. Yesen Domain — Transactions Page And Create Account Page

7.1 Feature Overview

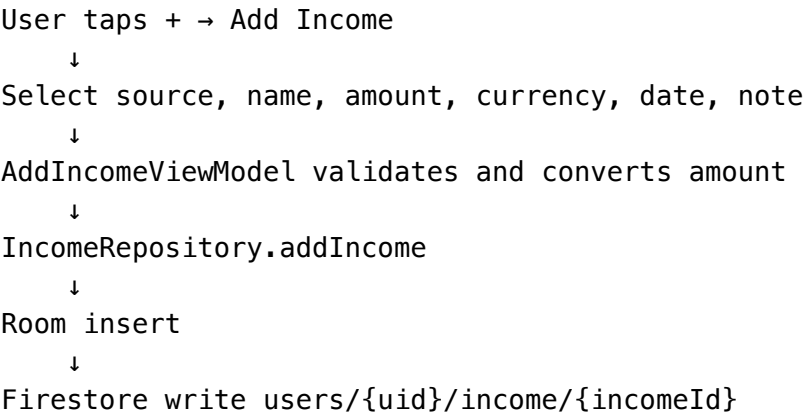
Yesen's domain covers:

- Create account flow.
- Add income.
- Add expense.
- History / transactions page.
- Transaction filtering and grouping.
- Transaction edit/delete.
- Income/expense repository and DAO communication.

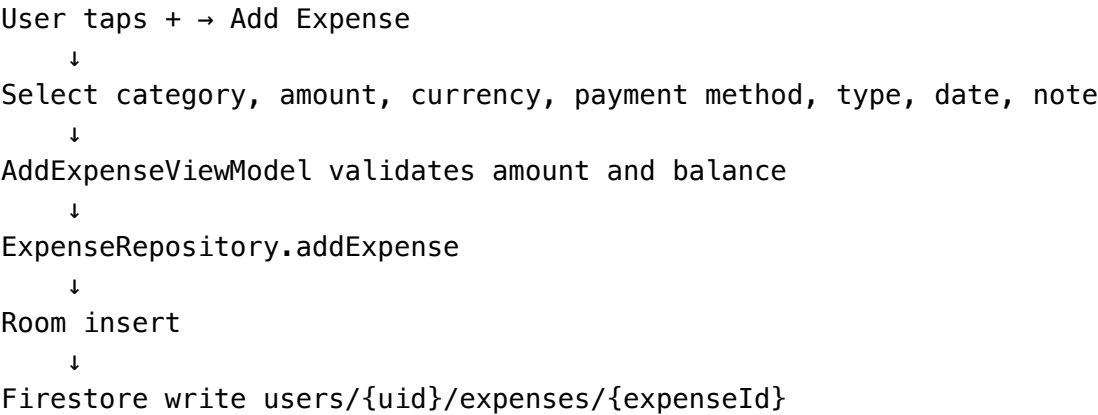
7.2 User Flow: Create Account



7.3 User Flow: Add Income



7.4 User Flow: Add Expense



7.5 Related Files

File	Purpose
ui/screen/CreateAccountScreen.kt	Register UI

File	Purpose
ui/viewmodel/CreateAccountViewModel.kt	Register validation/state
ui/screen/transactions/TransactionsScreen.kt	History list and filtering
ui/screen/transactions/AddIncomeScreen.kt	Add/edit income UI
ui/screen/transactions/AddExpenseScreen.kt	Add/edit expense UI
ui/viewmodel/TransactionsViewModel.kt	Transaction filter/group/delete state
ui/viewmodel/AddIncomeViewModel.kt	Income form state and save logic
ui/viewmodel/AddExpenseViewModel.kt	Expense form state and save logic
data/repository/TransactionRepository.kt	Combined transaction repository contract
data/repository/FirebaseTransactionRepository.kt	Combines income and expense flows
data/repository/IncomeRepository.kt	Income repository contract
data/repository/ExpenseRepository.kt	Expense repository contract
data/remote/IncomeRepositoryImpl.kt	Income Room/Firestore implementation
data/remote/ExpenseRepositoryImpl.kt	Expense Room/Firestore implementation
data/local/dao/IncomeDao.kt	Income SQL queries
data/local/dao/ExpenseDao.kt	Expense SQL queries
data/local/entity/IncomeEntryEntity.kt	Income Room schema
data/local/entity/ExpenseEntryEntity.kt	Expense Room schema
data/service/CurrencyRateService.kt	Fiat rate service
data/service/CryptoRateService.kt	Crypto rate service

7.6 Create Account File Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/ui/screen/CreateAccountScreen.kt

Purpose

Displays the registration form.

Important Inputs

- Full name.
- Email address.
- Password.
- Confirm password.
- Default currency dropdown.

Logic

1. User enters form values.
2. Screen sends changes to `CreateAccountViewModel`.
3. User taps Create Account.
4. `ViewModel` validates:
 - Name is not blank.
 - Email is not blank.
 - Password meets minimum requirements.
 - Confirm password matches.
 - Currency is selected.
5. `ViewModel` calls `AuthRepository.createAccount`.
6. On success, app session updates and user enters main app.

7.7 Create Account Backend Explanation

File Path

`app/src/main/kotlin/com/spendly/financetracker/data/repository/FirebaseAuthRepository.kt`

Important Function

`createAccount(name, email, password, defaultCurrency)`

Step-By-Step Logic

1. Firebase Auth creates user with email and password.
2. UID is read from Firebase user.
3. A `UserProfileEntity` is created with:
 - uid
 - name
 - email
 - defaultCurrency
 - timestamps
 - default theme mode
4. Profile is saved locally through `UserProfileDao`.
5. Profile is written to Firestore at `users/{uid}/profile/main`.
6. Local profile is marked as synced if Firestore succeeds.

7.8 Transactions Screen Explanation

File Path

`app/src/main/kotlin/com/spendly/financetracker/ui/screen/transactions/TransactionsScreen.kt`

Purpose

Displays the History page with transaction filtering.

Important Features

- Month picker.
- Filter chips: All, Expenses, Incomes.
- Grouped transaction list.
- Expandable transaction rows.
- Edit and delete actions.
- Delete confirmation dialog.

Step-By-Step Logic

1. Screen obtains `TransactionsViewModel` using Hilt.
2. It collects `TransactionsUiState`.
3. It displays selected month using shared `SpendlyMonthPicker`.
4. It displays filter chips.
5. It displays grouped transactions by date.
6. On edit, it routes to Add Income or Add Expense based on transaction type.
7. On delete, it opens confirmation dialog.
8. On confirmation, it calls `viewModel.delete(transaction)`.

7.9 Transactions ViewModel Explanation

File Path

`app/src/main/kotlin/com/spendly/financetracker/ui/viewmodel/TransactionsViewModel.kt`

Purpose

Keeps History screen logic out of the Composable.

Important Logic

- Observes current Firebase UID.
- Collects transaction Flow from `TransactionRepository`.
- Applies selected month filter.
- Applies selected type filter.
- Sorts by `dateMillis`.
- Groups transactions by display date.

- Deletes selected transaction through repository.

Why This Is MVVM-Correct

The Composable does not calculate the grouped list manually. It receives prepared state from the ViewModel.

7.10 Add Income UI Explanation

File Path

`app/src/main/kotlin/com/spendly/financetracker/ui/screen/transactions/AddIncomeScreen.kt`

Important Features

- Income source selector.
- Add custom income source.
- Hide/delete source settings.
- Combined currency dropdown and amount input.
- Date field using read-only clickable input style.
- Note field.
- Optional recurring toggle.
- Optional crypto-specific fields.
- Exchange rate status.

Currency Handling

The income form supports:

- Default currency.
- USD.
- If default currency is USD, also LKR.
- Manual exchange rate entry if selected currency differs from default currency.
- Crypto manual rate flow for unsupported coin/rate updates.

7.11 Add Income ViewModel Explanation

File Path

`app/src/main/kotlin/com/spendly/financetracker/ui/viewmodel/AddIncomeViewModel.kt`

Main Responsibilities

- Load existing transaction when editing.
- Validate amount/name/source/date.
- Calculate converted default-currency amount.
- Handle source settings.
- Call `IncomeRepository.addIncome` or `IncomeRepository.updateIncome`.

Validation Examples

Validation	Reason
Amount must be greater than zero	Prevents invalid finance records
Exchange rate required when currency differs	Needed to convert to default currency
Source required	Needed for analytics and filtering
Date must be selected	Needed for correct monthly grouping

7.12 Add Expense UI Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/ui/screen/transactions/AddExpenseScreen.kt

Important Features

- Category selector with icons.
- Custom category creation.
- Built-in category hide behavior.
- Fixed payment methods: Card, Cash, Auto-debit.
- Expense type selector: Committed or Discretionary.
- Combined currency dropdown and amount field.
- Date and note fields.

7.13 Add Expense ViewModel Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/ui/viewmodel/AddExpenseViewModel.kt

Main Responsibilities

- Validate expense amount.
- Validate exchange rate if needed.
- Validate available balance.
- Build TransactionDraft.
- Save expense through repository.
- Update category settings through profile repository.

7.14 Income Repository Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/data/remote/IncomeRepositoryImpl.kt

Important Functions

Function	Purpose
<code>observeIncome(uid)</code>	Observes local income rows
<code>getIncome(uid)</code>	Reads local income list
<code>addIncome(uid, draft)</code>	Creates income locally and remotely
<code>updateIncome(uid, transaction)</code>	Updates existing income
<code>deleteIncome(uid, id)</code>	Deletes income locally and remotely
<code>syncWithFirestore(uid)</code>	Pushes unsynced income and pulls remote income

Firestore Path

`users/{uid}/income/{incomeId}`

7.15 Expense Repository Explanation

File Path

`app/src/main/kotlin/com/spendly/financetracker/data/remote/ExpenseRepositoryImpl.kt`

Important Functions

Function	Purpose
<code>observeExpenses(uid)</code>	Observes local expenses
<code>addExpense(uid, draft)</code>	Creates expense locally/remotely
<code>updateExpense(uid, transaction)</code>	Updates expense
<code>deleteExpense(uid, id)</code>	Deletes expense
<code>syncWithFirestore(uid)</code>	Pushes unsynced and pulls remote expenses

Goal Expense Special Logic

If an expense is linked to a goal by `goalId`, deleting that transaction subtracts the amount from the goal. This is important because goal savings are stored as expense transactions with category `Goal`.

7.16 Yesen Viva Questions And Answers

Question	Answer
Why are income and expenses separate entities?	They have different fields and different Firestore subcollections, but they are combined for display through <code>TransactionRepository</code> .

Question	Answer
How does History show both income and expenses?	<code>FirestoreTransactionRepository</code> combines income and expense flows into <code>FinanceTransaction</code> rows.
Where is transaction filtering done?	In <code>TransactionsViewModel</code> , not directly in the UI.
What happens if Firestore save fails?	Room still contains the record with <code>isSynced=false</code> , and sync can retry later.
Why store <code>amountCents</code> as Long?	It avoids floating-point precision issues for money.

7.17 Demo Script For Yesen Domain

1. Open Create Account screen and explain field validation.
2. Sign in or create a test account.
3. Go to History.
4. Add an income record.
5. Add an expense record.
6. Show the list grouped by date.
7. Edit and delete a transaction.
8. Explain Room first, Firestore second, `isSynced` field, and `WorkManager` retry.

8. Nikini Domain — Goal Page And Login Page

8.1 Feature Overview

Nikini's domain covers:

- Login page.
- Goal tracker.
- Add goal.
- Edit goal.
- Goal details.
- Add savings.
- Goal icon suggestion and manual icon picker.
- Goal repository/backend logic.

8.2 Login User Flow

User opens Login

↓

User enters email and password

↓

`AuthScreen` sends event to `FinanceViewModel`

↓

`FinanceViewModel` validates basic fields

↓
 AuthRepository.signIn calls Firebase Auth
 ↓
 Session Flow emits Firebase UID
 ↓
 FinanceViewModel loads user data and starts sync
 ↓
 App navigates to Dashboard

8.3 Goal User Flow

User opens Goal Tracker
 ↓
 User taps + to add goal
 ↓
 AddGoalScreen collects title, target, date, initial saved, icon, primary flag
 ↓
 GoalsViewModel validates draft
 ↓
 GoalRepository.saveGoal saves goal locally/remotely
 ↓
 If initial saved > 0, linked Goal expense is created
 ↓
 Goal list updates through Room Flow

8.4 Related Files

File	Purpose
ui/screen/AuthScreen.kt	Login UI
ui/viewmodel/FinanceViewModel.kt	Login state/session handling
data/repository/AuthRepository.kt	Auth contract
data/repository/FirebaseAuthRepository.kt	Firebase Auth implementation
data/model/UserSession.kt	Session model
ui/screen/goals/GoalsScreen.kt	Main goal UI content
ui/screen/goals/GoalScreen.kt	Goal screen wrapper
ui/screen/goals/AddGoalScreen.kt	Add goal route wrapper
ui/screen/goals/EditGoalScreen.kt	Edit goal route wrapper
ui/screen/goals/GoalDetailsScreen.kt	Goal details route wrapper
ui/viewmodel/GoalsViewModel.kt	Goal state and business logic
data/model/SavingsGoal.kt	Goal domain model
data/local/entity/SavingsGoalEntity.kt	Room goal table
data/local/dao/GoalDao.kt	Goal SQL queries
data/repository/GoalRepository.kt	Goal repository contract
data/remote/GoalRepositoryImpl.kt	Goal Room/Firestore implementation

File	Purpose
ui/util/GoalIconUtils.kt	Goal icon suggestion and mapping

8.5 Login UI Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/ui/screen/AuthScreen.kt

Purpose

Displays login form.

Important UI Features

- Email field.
- Password field.
- Password visibility toggle.
- Forgot password action.
- Sign in button.
- Create account navigation.

Backend Connection

The login screen does not call Firebase directly. It calls ViewModel callbacks. FinanceViewModel calls AuthRepository.signIn, and FirebaseAuthRepository performs the Firebase operation.

8.6 Firebase Auth Repository Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/data/repository/FirebaseAuthRepository.kt

Login Function

signIn(email, password)

Step-By-Step Logic

1. Calls Firebase Auth sign-in with email and password.
2. Receives Firebase user.
3. Creates UserSession using UID and email.
4. Returns session result to ViewModel.
5. observeSession() also emits auth state changes.

Forgot Password

`sendPasswordResetEmail(email)` exists and calls Firebase password reset.

Change Password

`updatePassword(currentPassword, newPassword)` exists and supports Firebase reauthentication flow.

8.7 Goals UI Explanation

File Path

`app/src/main/kotlin/com/spendly/financetracker/ui/screen/goals/GoalsScreen.kt`

Purpose

This is the main goal UI file. It includes goal list, goal details, add/edit form content, add savings dialog, icon picker, and chart-like goal progress UI.

Important UI Sections

- Primary Goals section.
- Other Goals section.
- Achieved goals handling.
- Goal detail screen.
- Add goal form.
- Edit goal form.
- Add savings dialog.
- Monthly savings chart.

8.8 Goal Icon Logic

File Path

`app/src/main/kotlin/com/spendly/financetracker/ui/util/GoalIconUtils.kt`

Purpose

Suggests goal icons from goal title keywords and maps icon keys to Compose icons.

Example mappings:

Goal Name Keyword	Suggested Icon
Car, Vehicle, Bike	Transport
House, Home, Rent	Home
Travel, Trip, Vacation	Travel

Goal Name Keyword	Suggested Icon
Education, Course, University	School
Laptop, Computer, PC	Laptop
Phone, Mobile	Phone
Wedding, Gift	Heart/Gift
Emergency, Medical, Health	Health/Security
Savings, Money, Fund	Money
Shopping	Shopping
Business, Startup	Business
Other	Default goal/flag

8.9 Goals ViewModel Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/ui/viewmodel/GoalsViewModel.kt

Responsibilities

- Holds goal form state.
- Loads existing goal for edit/details.
- Validates title, amount, date, initial saved amount.
- Suggests icon when title changes.
- Allows manual icon override.
- Saves new/edited goals.
- Adds savings to goals.
- Prevents invalid over-saving.
- Deletes goals.

8.10 Goal Repository Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/data/remote/GoalRepositoryImpl.kt

Important Functions

Function	Purpose
observeGoals(uid)	Emits local goals from Room
getGoal(uid, id)	Reads one goal
saveGoal(uid, goal, balanceCents)	Saves new or edited goal
addSavings(uid, goalId, amountCents, dateMillis)	Adds money to goal

Function	Purpose
<code>deleteGoal(uid, goalId)</code>	Deletes goal locally/remotely
<code>syncWithFirestore(uid)</code>	Pushes unsynced goals and pulls remote goals

8.11 Goal Savings As Expense

The app records goal savings as expense transactions:

Field	Value
<code>category</code>	Goal
<code>name</code>	Goal title
<code>expenseType</code>	DISCRETIONARY
<code>goalId</code>	Linked goal ID
<code>amountCents</code>	Saved amount
<code>dateMillis</code>	Savings date

Why this matters:

- Dashboard net savings and balance reflect money moved into a goal.
- History can show goal savings as a transaction.
- If a linked expense is deleted, the goal saved amount is adjusted.

8.12 Goal Validation

Validation	Reason
Goal title cannot be blank	Need meaningful goal name
Target amount must be positive	Goal cannot target zero
Initial saved cannot exceed target	Prevents impossible progress
Added savings cannot exceed remaining amount	Prevents <code>savedCents > targetCents</code>
Savings cannot exceed available balance	Prevents invalid finance state

If savings exceeds the target, UI should show:

Amount exceed target value

8.13 Nikini Viva Questions And Answers

Question	Answer
How are goals stored?	In Room SavingsGoalEntity and Firestore <code>users/{uid}/goals/{goalId}</code> .
How is progress calculated?	<code>savedCents / targetCents</code> , usually displayed as a percentage and progress

Question	Answer
Why create an expense when adding goal savings?	bar. It keeps financial balance consistent because saved money is treated as money allocated out of available funds.
Can multiple primary goals exist?	The updated logic supports multiple primary goals by using the <code>isPrimary</code> flag.
Does Login screen call Firebase directly?	No. It calls <code>ViewModel</code> , which calls <code>AuthRepository</code> .

8.14 Demo Script For Nikini Domain

1. Show login screen.
2. Explain Firebase Auth sign-in and session Flow.
3. Navigate to Goals.
4. Create a goal and show automatic icon suggestion.
5. Manually change icon.
6. Add initial saved amount.
7. Open goal details and add savings.
8. Try adding too much and explain validation.
9. Show linked goal expense in History.

9. Mahima Domain — Analytics Page, Room DB, Firestore, Sync

9.1 Feature Overview

Mahima's domain covers:

- Analytics page.
- Room database setup.
- Entities and DAOs.
- Firestore initialization/access setup.
- Caching strategy.
- Sync logic.
- WorkManager.

9.2 Analytics User Flow

User opens Analytics

↓

`AnalyticsViewModel` gets current UID

↓

`TransactionRepository` observes income + expenses from Room

↓

`AnalyticsViewModel` filters selected month

↓
 AnalyticsViewModel calculates totals, category percentages, split, overview
 ↓
 AnalyticsScreen displays cards and charts

9.3 Related Files

File	Purpose
ui/screen/analytics/AnalyticsScreen.kt	Analytics UI
ui/viewmodel/AnalyticsViewModel.kt	Analytics calculations
ui/components/SpendlyMonthPicker.kt	Shared month selector
ui/util/MonthOptions.kt	Calendar-backed month option model
ui/util/UiUtils.kt	Money/date formatting
data/local/db/SpendlyDatabase.kt	Room database and migrations
data/local/entity/*.kt	Room table schemas
data/local/dao/*.kt	Room queries
di/AppModule.kt	Database/Firebase providers
data/firebase/FirebaseBootstrap.kt	Firebase setup status helper
worker/SpendlySyncWorker.kt	Background sync worker
util/SyncManager.kt	Sync scheduling
firestore.rules	Firestore rules file
firestore.indexes.json	Firestore indexes file

9.4 Analytics UI Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/ui/screen/analytics/AnalyticsScreen.kt

Purpose

Displays analytics in a visual dashboard style.

UI Sections

- Title row with month picker.
- Total Income tile.
- Total Expenses tile.
- Spending by Category card.
- Donut chart.
- Category legend with percentages.
- Committed vs Discretionary card.
- Monthly Overview card.

- Income Sources card.

Important Composables

Composable	Responsibility
AnalyticsScreen	Main analytics page
SummaryTiles	Income/expense summary
SpendingByCategoryCard	Category breakdown
DonutChart	Canvas donut chart
SpendingSplitCard	Committed/discretionary comparison
MonthlyOverviewCard	5-month income/expense bars
IncomeSourcesCard	Income source breakdown

9.5 Analytics ViewModel Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/ui/viewmodel/AnalyticsViewModel.kt

Purpose

Moves analytics calculations out of the Composable.

Important Calculations

Calculation	Explanation
Selected month income	Sum income transactions where dateMillis is in selected month
Selected month expenses	Sum expense transactions where dateMillis is in selected month
Category breakdown	Group expenses by category and calculate percentages
Spending split	Separate committed and discretionary expenses
Monthly overview	Last 5 months relative to selected month
Income sources	Group income transactions by source

9.6 Room DB Setup Explanation

File Path

app/src/main/kotlin/com/spendly/financetracker/data/local/db/SpendlyDatabase.kt

Purpose

Defines the Room database, entities, DAOs, and migrations.

Current Version

Room database version: 5

Entities Included

- IncomeEntryEntity
- ExpenseEntryEntity
- SavingsGoalEntity
- UserProfileEntity

Migrations

Migration	Purpose
1 -> 2	Adds currency, crypto, profile image/settings, and initial goal fields
2 -> 3	Adds categorySettingsJson
3 -> 4	Adds goal iconKey
4 -> 5	Adds profile themeMode

9.7 Firestore Setup Explanation

Firestore is provided through Hilt in `di/AppModule.kt`.

Configured Firestore behavior:

- Local persistent cache is enabled.
- Cache size is set to unlimited in the Firestore settings.

Important note:

`firestore.rules` exists, but it appears to reference an older transaction schema using `users/{userId}/transactions/{transactionId}`. Current code uses `income`, `expenses`, `goals`, and `profile` subcollections. Therefore:

Firestore security rules matching the current implemented schema: Not found in current codebase.

9.8 Sync Logic Explanation

Files

- `app/src/main/kotlin/com/spendly/financetracker/worker/SpendlySyncWorker.kt`
- `app/src/main/kotlin/com/spendly/financetracker/util/SyncManager.kt`

- `app/src/main/kotlin/com/spendly/financetracker/SpendlyApplication.kt`

Sync Process

WorkManager starts SpendlySyncWorker

↓

Worker reads current UID from AuthRepository

↓

If no UID, worker returns success

↓

Worker launches sync for:

- User profile
- Income
- Expenses
- Goals

↓

Each repository pushes local unsynced rows

↓

Each repository pulls remote Firestore docs

↓

Newest updatedAtMillis wins where merge exists

9.9 Caching Strategy

The app uses two caching levels:

1. Firestore local persistence cache.
2. Room database cache.

Room is the main app-facing cache. Compose screens observe Room-backed flows through repositories and ViewModels. Firestore is the remote sync target.

9.10 Mahima Viva Questions And Answers

Question	Answer
Why use Room if Firestore has offline cache?	Room gives structured SQL queries, observable local state, migrations, and an app-owned cache.
What does WorkManager sync?	Profile, income, expenses, and goals for the current Firebase UID.
How does Analytics get data?	It observes combined transactions from the repository and aggregates them in AnalyticsViewModel.
Are Firestore indexes configured?	<code>firestore.indexes.json</code> exists but contains no custom indexes.
Are current Firestore rules complete?	No. The rules file appears to match an older transaction schema, not the current subcollections.

9.11 Demo Script For Mahima Domain

1. Open Analytics.
2. Change month using month picker.
3. Show totals and donut chart.
4. Explain calculations are in AnalyticsViewModel.
5. Explain Room entities and DAOs.
6. Explain Firestore structure under users/{uid}.
7. Explain WorkManager periodic sync and offline-first behavior.

10. Database And Schema Explanation

10.1 Room Entity Summary

Entity	Table	Primary Key	Purpose	Relat
IncomeEntryEntity	income_entries	id	Stores income records	IncomeEntr IncomeDao.
ExpenseEntryEntity	expense_entries	id	Stores expense records	ExpenseEnt ExpenseDao
SavingsGoalEntity	savings_goals	id	Stores savings goals	SavingsGoa GoalDao.kt
UserProfileEntity	user_profiles	uid	Stores profile/settings	UserProfil UserProfil

10.2 Income Schema

Field	Type	Purpose
id	String	Income ID
userId	String	Firebase UID owner
name	String	Income display name
amountCents	Long	Converted/default-currency amount in cents
source	String	Salary, freelance, crypto, etc.
dateMillis	Long	User-selected income date
note	String	Optional note
isSynced	Boolean	Whether Firestore upload succeeded
createdAtMillis	Long	Creation audit timestamp
updatedAtMillis	Long	Last update timestamp
originalAmount	Double?	Original user-entered amount
originalCurrency	String?	Currency selected by user
defaultCurrency	String?	User default currency at save time
exchangeRate	Double?	Rate used for conversion

Field	Type	Purpose
isRecurring	Boolean	Recurring income toggle
cryptoCoin	String?	Crypto coin name/symbol
cryptoAmount	Double?	Crypto quantity
cryptoRate	Double?	Crypto rate used
cryptoRateSource	String?	Rate source
cryptoRateFetchedAt	Long?	Rate fetch timestamp

10.3 Expense Schema

Field	Type	Purpose
id	String	Expense ID
userId	String	Firebase UID owner
name	String	Expense display name
amountCents	Long	Converted/default-currency expense amount
category	String	Expense category
dateMillis	Long	User-selected expense date
note	String	Optional note
isSynced	Boolean	Whether Firestore upload succeeded
createdAtMillis	Long	Creation audit timestamp
updatedAtMillis	Long	Last update timestamp
originalAmount	Double?	Original entered amount
originalCurrency	String?	Selected currency
defaultCurrency	String?	User default currency
exchangeRate	Double?	Rate used for conversion
paymentMethod	String?	Card, Cash, Auto-debit
expenseType	String?	Committed or Discretionary
goalId	String?	Linked goal if category is Goal

10.4 Goal Schema

Field	Type	Purpose
id	String	Goal ID
userId	String	Firebase UID owner
title	String	Goal name
status	String	Tracking, achieved, etc.
targetCents	Long	Target amount
savedCents	Long	Saved amount
dueDateMillis	Long	Target date
category	String	Goal category

Field	Type	Purpose
isPrimary	Boolean	Whether shown as primary goal
isSynced	Boolean	Whether Firestore upload succeeded
createdAtMillis	Long	Creation timestamp
updatedAtMillis	Long	Last update timestamp
initialSavedCents	Long	Initial saved amount
defaultCurrency	String?	Currency
iconKey	String	Goal icon key

10.5 User Profile Schema

Field	Type	Purpose
uid	String	Firebase UID
name	String	User name
email	String	User email
defaultCurrency	String	Main currency
createdAtMillis	Long	Creation timestamp
updatedAtMillis	Long	Last update timestamp
isSynced	Boolean	Sync state
profileImageUri	String?	Local image URI
exchangeRateSettings	String?	Stored settings
notificationFrequency	String?	Notification setting if used
reminderTime	String?	Reminder time if used
categorySettingsJson	String?	Custom/hidden categories and sources
themeMode	String	SYSTEM, LIGHT, DARK

10.6 Firestore Collection Summary

Collection / Document	Purpose	Related Repository
users/{uid}/profile/main	User profile and settings	UserRepositoryImpl, FirebaseAuthRepository
users/{uid}/income/{incomeId}	Income records	IncomeRepositoryImpl
users/{uid}/expenses/{expenseId}	Expense records	ExpenseRepositoryImpl
users/{uid}/goals/{goalId}	Savings goals	GoalRepositoryImpl

10.7 Relationships

Relationship	How Implemented
User to income	IncomeEntryEntity.userId and Firestore path users/{uid}/income
User to expense	ExpenseEntryEntity.userId and Firestore path users/{uid}/expenses
User to goal	SavingsGoalEntity.userId and Firestore path users/{uid}/goals
Goal to goal-saving expense	ExpenseEntryEntity.goalId references SavingsGoalEntity.id
Profile to category settings	UserProfileEntity.categorySettingsJson stores custom/hidden settings
Profile to theme	UserProfileEntity.themeMode

10.8 Why Amounts Are Stored As Cents

Money is stored as Long cents (amountCents, targetCents, savedCents) to avoid floating-point rounding problems. For example, storing 10.10 as a Double can cause precision issues, but storing 1010 cents as a Long is exact.

11. Firestore Queries And Backend Logic

11.1 Profile Create/Save

Detail	Value
File	data/repository/FirebaseAuthRepository.kt, data/remote/UserRepositoryImpl.kt
Function	createAccount, upsertProfile, syncOne
Firestore path	users/{uid}/profile/main
Purpose	Create or update user profile
Error handling	If write fails after local save, profile remains unsynced

11.2 Profile Fetch / Sync

Detail	Value
File	data/remote/UserRepositoryImpl.kt
Function	syncWithFirestore(uid)
Firestore path	users/{uid}/profile/main
Purpose	Merge remote profile with local profile

Detail	Value
Conflict handling	Prefer newest updatedAtMillis between local and remote

11.3 Add Income

Detail	Value
File	data/remote/IncomeRepositoryImpl.kt
Function	addIncome(uid, draft)
Firestore path	users/{uid}/income/{incomeId}
Purpose	Save income locally and remotely
Error handling	Local row remains isSynced=false if Firestore fails

11.4 Update Income

Detail	Value
File	data/remote/IncomeRepositoryImpl.kt
Function	updateIncome(uid, transaction)
Firestore path	users/{uid}/income/{incomeId}
Purpose	Update existing income
Error handling	Local update still exists for later sync

11.5 Delete Income

Detail	Value
File	data/remote/IncomeRepositoryImpl.kt
Function	deleteIncome(uid, id)
Firestore path	users/{uid}/income/{incomeId}
Purpose	Remove income
Limitation	Offline-safe delete tombstones are not found in current codebase

11.6 Income Sync

Detail	Value
File	data/remote/IncomeRepositoryImpl.kt
Function	syncWithFirestore(uid)
Query	Reads all documents under users/{uid}/income
Purpose	Push unsynced local income and pull remote income
Conflict handling	Newer updatedAtMillis wins

11.7 Add Expense

Detail	Value
File	data/remote/ExpenseRepositoryImpl.kt
Function	addExpense(uid, draft)
Firestore path	users/{uid}/expenses/{expenseId}
Purpose	Save expense locally and remotely
Error handling	Local row remains unsynced if remote save fails

11.8 Update Expense

Detail	Value
File	data/remote/ExpenseRepositoryImpl.kt
Function	updateExpense(uid, transaction)
Firestore path	users/{uid}/expenses/{expenseId}
Purpose	Update expense
Error handling	Unsynced local update can be retried

11.9 Delete Expense

Detail	Value
File	data/remote/ExpenseRepositoryImpl.kt
Function	deleteExpense(uid, id)
Firestore path	users/{uid}/expenses/{expenseId}
Special logic	If linked to goal, subtract amount from goal saved total
Limitation	Tombstone-based remote delete sync is not found in current codebase

11.10 Expense Sync

Detail	Value
File	data/remote/ExpenseRepositoryImpl.kt
Function	syncWithFirestore(uid)
Query	Reads all documents under users/{uid}/expenses
Purpose	Push local unsynced expenses and pull remote expenses

11.11 Save Goal

Detail	Value
File	data/remote/GoalRepositoryImpl.kt

Detail	Value
Function	<code>saveGoal(uid, goal, balanceCents)</code>
Firestore path	<code>users/{uid}/goals/{goalId}</code>
Purpose	Create or update savings goal
Special logic	Initial saved amount creates a linked goal expense

11.12 Add Goal Savings

Detail	Value
File	<code>data/remote/GoalRepositoryImpl.kt</code>
Function	<code>addSavings(uid, goalId, amountCents, dateMillis)</code>
Firestore path	Updates <code>users/{uid}/goals/{goalId}</code> and creates expense
Purpose	Increase goal saved amount and record money movement
Validation	Prevents exceeding target

11.13 Delete Goal

Detail	Value
File	<code>data/remote/GoalRepositoryImpl.kt</code>
Function	<code>deleteGoal(uid, goalId)</code>
Firestore path	<code>users/{uid}/goals/{goalId}</code>
Purpose	Remove goal
Limitation	Cascading remote cleanup of linked goal expenses is not clearly implemented

11.14 Goal Sync

Detail	Value
File	<code>data/remote/GoalRepositoryImpl.kt</code>
Function	<code>syncWithFirestore(uid)</code>
Query	Reads all documents under <code>users/{uid}/goals</code>
Purpose	Push local unsynced goals and pull remote goals

11.15 Firestore Security Rules Status

`firestore.rules` exists, but the current rule content appears to reference an older path:

`users/{userId}/transactions/{transactionId}`

Current app code uses:

users/{uid}/income
 users/{uid}/expenses
 users/{uid}/goals
 users/{uid}/profile/main

Therefore:

Firestore security rules for the current implemented schema: Not found in current codebase.

This is a risk area lecturers may ask about.

12. Room Database Logic

12.1 Room File Table

Room File	Type	Purpose	Important Methods	Used
SpendlyDatabase.kt	Database	Defines entities, DAOs, migrations	incomeDao, expenseDao, goalDao, userProfileDao	Hilt AppModule
IncomeEntryEntity.kt	Entity	Income table	Fields and indices	IncomeDao, IncomeRepos
ExpenseEntryEntity.kt	Entity	Expense table	Fields and indices	ExpenseDao, ExpenseRepo
SavingsGoalEntity.kt	Entity	Goal table	Fields and indices	GoalDao, GoalReposit
UserProfileEntity.kt	Entity	Profile table	Fields and sync settings	UserProfile UserReposit
IncomeDao.kt	DAO	Income SQL	observeByUser, getUnsynced, markAsSynced	Income reposi
ExpenseDao.kt	DAO	Expense SQL	observeByUser, getUnsynced, markAsSynced	Expense repos
GoalDao.kt	DAO	Goal SQL	observeByUser, getById, markAsSynced	Goal repositor
UserProfileDao.kt	DAO	Profile SQL	upsert, observeById, getUnsynced	User repositor

12.2 Room Database Class

File Path

`app/src/main/kotlin/com/spendly/financetracker/data/local/db/SpendlyDatabase.kt`

Important Points

- Annotated with `@Database`.
- Uses version 5.
- Contains four entities.
- Exposes four abstract DAO functions.
- Includes non-destructive migrations.

12.3 DAO Pattern

Each DAO follows a similar structure:

- Observe all records by user ID using `Flow`.
- Get one record by ID.
- Insert or update records.
- Delete records by ID.
- Find unsynced records.
- Mark records as synced after Firestore success.

12.4 Local Cache Behavior

The app usually writes to Room first:

1. Generate or receive model data.
2. Convert model to Room entity.
3. Save entity with `isSynced=false`.
4. Try remote Firestore write.
5. If successful, mark as synced.
6. If failed, keep record local for retry.

12.5 Type Converters

Room `TypeConverter` files: Not found in current codebase.

This is acceptable because current Room fields are primitive types such as `String`, `Long`, `Boolean`, and nullable primitive/string fields.

12.6 Room Schema Export

The database uses `exportSchema = false`. This means Room schema JSON export is not enabled.

Potential improvement:

Enable schema export for production-grade migration auditing.

13. Hilt Dependency Injection Explanation

13.1 Why Hilt Is Used

Hilt manages object creation and dependencies. Without Hilt, every ViewModel or repository would need manual constructors and factories. Hilt helps keep code cleaner and makes testing easier.

13.2 Application Class

File Path

```
app/src/main/kotlin/com/spendly/financetracker/SpendlyApplication.kt
```

Important Annotations

```
@HiltAndroidApp
```

Purpose

- Enables Hilt for the app.
- Injects HiltWorkerFactory.
- Injects SyncManager.
- Provides WorkManager configuration.
- Schedules periodic sync in onCreate.

13.3 Main Activity

File Path

```
app/src/main/kotlin/com/spendly/financetracker/MainActivity.kt
```

Important Annotation

```
@AndroidEntryPoint
```

Purpose

Allows Hilt to inject dependencies into Compose through `hiltViewModel()`.

13.4 App Module

File Path

`app/src/main/kotlin/com/spendly/financetracker/di/AppModule.kt`

Provides

Dependency	Purpose
FirebaseAuth	Authentication
Firestore	Cloud database
SpendlyDatabase	Local Room database
IncomeDao	Income SQL
ExpenseDao	Expense SQL
GoalDao	Goal SQL
UserProfileDao	Profile SQL
WorkManager	Background work

13.5 Repository Module

File Path

`app/src/main/kotlin/com/spendly/financetracker/di/RepositoryModule.kt`

Purpose

Binds repository interfaces to their implementations.

Interface	Implementation
AuthRepository	FirebaseAuthRepository
IncomeRepository	IncomeRepositoryImpl
ExpenseRepository	ExpenseRepositoryImpl
GoalRepository	GoalRepositoryImpl
UserRepository	UserRepositoryImpl
TransactionRepository	FirebaseTransactionRepository

13.6 ViewModel Injection

ViewModels use `@HiltViewModel` and constructor injection.

Example dependencies:

- `FinanceViewModel` receives auth, repositories, sync manager.
- `TransactionsViewModel` receives auth and transaction repository.
- `AnalyticsViewModel` receives auth and transaction repository.

- `GoalsViewModel` receives auth and goal repository.

13.7 Worker Injection

`SpendlySyncWorker` uses `@HiltWorker` so repositories can be injected into background work.

14. WorkManager And Sync Explanation

14.1 Why WorkManager Is Used

`WorkManager` is used because sync must happen reliably in the background, even if the app is not actively in the foreground. It supports network constraints and retry policies.

14.2 Worker File

File Path

```
app/src/main/kotlin/com/spendly/financetracker/worker/SpendlySyncWorker.kt
```

Important Class

`SpendlySyncWorker`

Important Function

```
doWork()
```

14.3 Sync Steps

```
doWork()  
  ↓  
Get current uid from AuthRepository  
  ↓  
If uid is null, return Result.success()  
  ↓  
Run sync jobs:  
  - userRepository.syncWithFirestore(uid)  
  - incomeRepository.syncWithFirestore(uid)  
  - expenseRepository.syncWithFirestore(uid)  
  - goalRepository.syncWithFirestore(uid)  
  ↓  
If all succeed, return Result.success()  
  ↓  
If error and attempt < 3, return Result.retry()
```

↓
 Otherwise return Result.failure()

14.4 Scheduling Logic

File Path

app/src/main/kotlin/com/spendly/financetracker/util/SyncManager.kt

Functions

Function	Purpose
schedulePeriodicSync()	Schedules periodic sync every 15 minutes
startImmediateSync()	Starts one-time sync immediately

14.5 Network Constraints

The worker request uses network constraints so sync runs when the network is connected.

14.6 Offline-First Behavior

The code supports partial offline-first behavior:

- Writes save locally first.
- Failed Firestore writes leave data unsynced.
- WorkManager can retry upload.

Limitations:

- Robust delete tombstones are not found.
- Full two-way conflict resolution for every edge case is limited.
- Remote deletion propagation to local Room is not clearly implemented.

14.7 Viva Questions

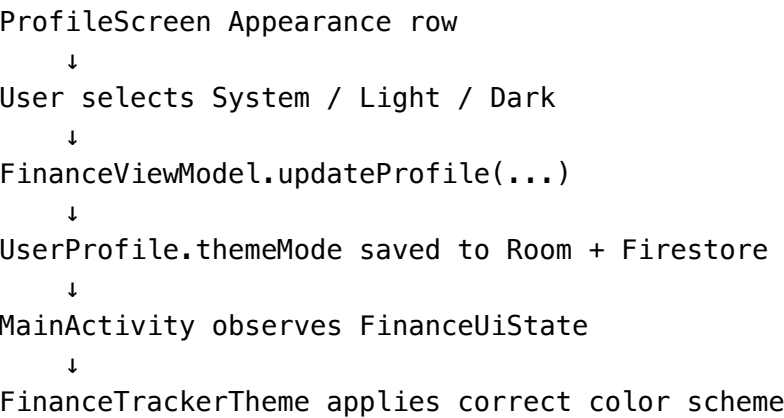
Question	Answer
Why not sync only when user opens app?	WorkManager allows retry in background with network constraints.
What data is synced?	Profile, income, expenses, and goals.
What is the minimum periodic interval?	WorkManager periodic work minimum is 15 minutes.
What happens if sync fails?	Worker retries up to configured attempts, then fails. Local unsynced data remains.

15. Theme, UI Consistency, And Design System

15.1 Theme Files

File	Purpose
ui/theme/Color.kt	Spendly brand and semantic colors
ui/theme/Theme.kt	Material light/dark color schemes
ui/theme/ThemeMode.kt	SYSTEM/LIGHT/DARK enum
ui/theme/Type.kt	Typography

15.2 Theme Mode Flow



15.3 Material Design 3 Usage

The app uses Material 3 components such as:

- Scaffold
- NavigationBar
- TopAppBar
- Card
- Button
- OutlinedTextField
- AlertDialog
- FloatingActionButton
- FilterChip

15.4 Shared UI Components

Component	File	Purpose
Bottom navigation	AppBottomNavigation.kt	Shared bottom tab UI
Add action FAB	SpendlyAddActionMenu.kt	Consistent plus button/menu
Month picker	SpendlyMonthPicker.kt	Shared History/Analytics month selector

Component	File	Purpose
Transaction row	TransactionListItem.kt	Consistent transaction display
Summary cards	SummaryCard.kt, SummaryPanel.kt	Dashboard summary UI
Goal card	GoalCard.kt	Goal summary UI
Design tokens	SpendlyDesignTokens.kt, SpendlyDesign.kt	Shared spacing, sizing, styles

15.5 UI Consistency Improvements Already Present

- Shared month picker for History and Analytics.
- Shared plus FAB overlay in app scaffold.
- Modern navigation transitions.
- Material color scheme support for dark/light/system modes.
- Central money/date formatting helpers.
- Shared transaction row component.

15.6 UI Risk Areas

Potential areas to test before demo:

- Dark mode contrast on all custom cards.
- Profile image picker permissions on different Android versions.
- FAB position across Home, History, and Goals.
- Analytics chart readability with empty data.
- Long text handling in goal and transaction names.

16. Main Functional Logic To Understand

16.1 Authentication Flow

```

AuthScreen
  ↓
FinanceViewModel.submitAuth
  ↓
AuthRepository.signIn
  ↓
FirebaseAuthRepository.signIn
  ↓
Firebase Auth
  ↓
observeSession emits UserSession
  ↓
FinanceViewModel observes user data

```

Must know:

- Firebase UID is the owner key.
- Session changes drive navigation.
- Login errors are stored in ViewModel state.

16.2 Registration Flow

```
CreateAccountScreen
    ↓
CreateAccountViewModel
    ↓
AuthRepository.createAccount
    ↓
Firebase Auth user created
    ↓
UserProfileEntity created
    ↓
Room profile saved
    ↓
Firestore profile saved
```

Must know:

- Default currency is saved during account creation.
- Profile is created under users/{uid}/profile/main.

16.3 Dashboard Calculation Flow

FinanceViewModel combines:

- Profile Flow
- Transaction Flow
- Goal Flow

↓

Calculates:

- balance
- monthly income
- monthly expense
- savings rate
- yearly savings
- recent transactions
- primary goal

↓

HomeScreen renders state

16.4 Transaction Add Flow

AddIncome/AddExpense Screen

↓

Screen-specific ViewModel validates

↓

Repository saves to Room

↓
Repository tries Firestore
↓
DAO marks synced on success

16.5 Transaction Update/Delete Flow

History row expanded
↓
Edit navigates to correct form
↓
Delete opens confirmation
↓
ViewModel calls repository delete
↓
Room and Firestore delete attempted

16.6 Goal Progress Flow

Goal savedCents and targetCents
↓
 $\text{progress} = \text{savedCents} / \text{targetCents}$
↓
UI progress bar and percentage

16.7 Goal Savings Flow

User adds savings
↓
Validate amount $\leq$ remaining target
↓
Update goal.savedCents
↓
Create linked expense with category Goal
↓
Sync goal and expense

16.8 Analytics Aggregation Flow

Transactions from Room
↓
Filter by selected month
↓
Group expenses by category
↓
Calculate percentages
↓
Split committed/discretionary
↓

Build 5-month overview

↓

AnalyticsScreen renders charts

16.9 Sync Flow

Local writes create isSynced=false rows

↓

Immediate sync or WorkManager periodic sync

↓

Repository pushes unsynced local rows

↓

Repository pulls remote rows

↓

Newest updatedAtMillis selected during merge

16.10 Currency / Exchange Rate Flow

User selects currency

↓

If selected currency == default currency:

 exchange field hidden

Else:

 exchange rate required or updated from service

↓

Converted amount stored in amountCents

Original amount/currency/rate stored for reference

Important:

API failure does not block saving if manual rate is valid.

17. Live Demo Presentation Flow

Minute 0-1: Project Introduction

What to click:

- Open app from launcher.
- Show splash/login/dashboard depending on test state.

What to say:

“Spendly is a personal finance tracker built with Kotlin, Compose, MVVM, Room, Firebase, Hilt, and WorkManager. It supports account-based finance tracking, local cache, cloud sync, dashboard summaries, goals, analytics, and profile settings.”

Technical point:

- Local-first architecture and Firebase UID ownership.

Possible lecturer question:

“What makes this different from a static UI app?”

Answer:

“The app has real local and remote data layers. Screens are backed by ViewModels, repositories, Room DAOs, and Firestore documents.”

Minute 1-3: Architecture Explanation

What to show:

- Briefly mention code structure or architecture diagram.

What to say:

“The data flow is Compose UI to ViewModel to Repository to Room and Firestore. Room provides local flows for the UI. Firestore provides cross-device persistence. Hilt injects all dependencies.”

Technical point:

- MVVM separation.
- Room and Firestore coordination.
- WorkManager sync.

Possible lecturer question:

“Why use repository interfaces?”

Answer:

“They keep ViewModels independent from concrete Firebase/Room classes and make the architecture cleaner and more testable.”

Minute 3-5: Register/Login Flow

What to click:

- Login or create account.
- Show forgot password if needed.

What to say:

“Authentication uses Firebase Auth. Registration also creates a profile document and local Room profile.”

Technical point:

- FirebaseAuthRepository.
- UserProfileEntity.
- Firestore path users/{uid}/profile/main.

Possible question:

“How do you prevent data mixing between users?”

Answer:

“Every local entity has `userId`, and every Firestore collection is nested under the Firebase UID.”

Minute 5-7: Dashboard Explanation

What to click:

- Open Home/Dashboard.

What to say:

“Dashboard is calculated from transaction and goal data. It displays total balance, monthly income, monthly expenses, net savings, savings rate, yearly savings, primary goals, and recent transactions.”

Technical point:

- Uses `dateMillis` for finance reporting.
- Uses money stored as cents.

Possible question:

“Where are these calculations done?”

Answer:

“They are prepared in ViewModel state, mainly `FinanceViewModel`, then passed to `HomeScreen`.”

Minute 7-9: Transactions Flow

What to click:

- Tap plus.
- Add income.
- Add expense.
- Open History.
- Filter income/expense.
- Edit/delete a transaction.

What to say:

“Transactions are separated into income and expense tables but combined into one UI list. Add and edit screens validate data and save through repositories.”

Technical point:

- Room insert first.
- Firestore sync second.

- `isSynced` flag.

Possible question:

“What if internet is unavailable?”

Answer:

“Local Room write can still succeed. The row remains unsynced and `WorkManager` can retry later.”

Minute 9-11: Goals Flow

What to click:

- Open Goal tab.
- Add goal.
- Show icon suggestion.
- Add savings.

What to say:

“Goals track a target amount and saved amount. Adding savings updates the goal and creates a linked expense transaction with category Goal.”

Technical point:

- `goalId` links expense to goal.
- Validation prevents saving more than target.

Possible question:

“Why is goal saving an expense?”

Answer:

“It represents money allocated out of available balance, so the dashboard and history remain consistent.”

Minute 11-13: Analytics/Profile/Settings

What to click:

- Open Analytics.
- Change month.
- Open Profile.
- Change appearance or currency.

What to say:

“Analytics is based on live transactions. It calculates selected-month totals, category percentages, spending split, and 5-month overview in `AnalyticsViewModel`. Profile stores settings like currency and theme.”

Technical point:

- Canvas donut chart.
- Profile synced through Firestore.
- Theme mode is user-specific.

Possible question:

“Are analytics hardcoded?”

Answer:

“No. They are calculated from transaction records observed through Room.”

Minute 13-15: Database, Sync, Conclusion

What to show:

- Explain schema and sync.

What to say:

“Room database version 5 contains income, expenses, goals, and profile tables. Firestore stores matching subcollections under each UID. WorkManager periodically syncs unsynced data when the network is connected.”

Technical point:

- Migrations.
- Hilt-provided DAOs/repositories.
- Sync worker.

Possible question:

“What is a current limitation?”

Answer:

“Firestore rules need updating to fully match the current income/expenses/goals/profile schema, and delete tombstones would improve offline delete sync.”

18. Viva / Engineering Discussion Questions And Answers

A. General Project Questions

Question	Short Answer
What is Spendly?	A personal finance tracker for income, expenses, savings goals, analytics, and profile settings.
What problem does it solve?	It helps users record money movement and understand savings and spending

Question	Short Answer
	patterns.
Is this app only UI?	No. It includes Room, Firebase Auth, Firestore, Hilt, WorkManager, and ViewModel state.
What is the main technical strength?	Local-first architecture with Room and Firestore sync.
Who are target users?	Students, freelancers, and individuals tracking personal finances.

B. Architecture Questions

Question	Short Answer
What architecture does the app use?	MVVM with repository pattern.
What is the data flow?	UI -> ViewModel -> Repository -> Room/Firestore -> Flow -> ViewModel state -> UI.
Why use repositories?	To hide data source details from ViewModels and keep code testable.
Why use local-first?	It improves responsiveness and allows offline-created records.
Where is navigation handled?	Mainly in <code>ui/FinanceTrackerApp.kt</code> .

C. MVVM Questions

Question	Short Answer
What is the View in this app?	Compose screens and components.
What is the ViewModel?	Classes in <code>ui/viewmodel</code> .
What is the Model?	Domain models and Room entities in <code>data/model</code> and <code>data/local/entity</code> .
Do Composables call repositories directly?	Important screens are designed to use ViewModels/callbacks instead.
Why is StateFlow useful?	It exposes observable state that Compose can collect and render.

D. ViewModel Questions

Question	Short Answer
What does <code>FinanceViewModel</code> do?	Handles session, app state, profile, dashboard data, and sync triggers.
What does <code>TransactionsViewModel</code> do?	Filters, groups, and deletes transactions.
What does <code>AnalyticsViewModel</code> do?	Calculates totals, category breakdown, split, and monthly overview.

Question	Short Answer
What does GoalsViewModel do?	Handles goal form state, validation, details, and savings.
Why not calculate analytics in Compose?	Business calculations should stay in ViewModel for MVVM separation.

E. Firestore Questions

Question	Short Answer
What is the Firestore root path?	users/{uid}.
Where is profile stored?	users/{uid}/profile/main.
Where is income stored?	users/{uid}/income/{incomeId}.
Where are expenses stored?	users/{uid}/expenses/{expenseId}.
Where are goals stored?	users/{uid}/goals/{goalId}.
How are conflicts handled?	Repositories generally compare updatedAtMillis and keep the newest.
Are security rules complete?	Current matching rules for the implemented schema are not found; existing rules look older.

F. Room DB Questions

Question	Short Answer
Why use Room?	For structured local cache, offline state, and Flow-based queries.
What is the database version?	Version 5.
What tables exist?	Income, expenses, goals, and user profiles.
Why migrations?	To preserve existing user data when schema changes.
Why isSynced?	To know which local rows need Firestore upload.

G. Hilt Questions

Question	Short Answer
Why use Hilt?	To inject dependencies and avoid manual factories.
Where is Hilt enabled?	SpendlyApplication.kt with @HiltAndroidApp.
Where are dependencies provided?	di/AppModule.kt and di/RepositoryModule.kt.
How are ViewModels injected?	With @HiltViewModel and hiltViewModel().

Question	Short Answer
How is WorkManager injected?	Through HiltWorkerFactory and @HiltWorker.

H. WorkManager Questions

Question	Short Answer
What does WorkManager do here?	Periodically syncs local and remote data.
Which file has the worker?	worker/SpendlySyncWorker.kt.
How often does periodic sync run?	Every 15 minutes.
What constraint is used?	Network connected.
What data is synced?	Profile, income, expenses, and goals.

I. Authentication Questions

Question	Short Answer
What auth provider is used?	Firebase Authentication.
What happens after login?	Session updates, data is observed, and immediate sync starts.
Does create account create profile data?	Yes, it creates a Room profile and Firestore profile document.
Is forgot password implemented?	sendPasswordResetEmail(email) exists in FirebaseAuthRepository.
Is password change implemented?	updatePassword exists with reauthentication support.

J. Transactions Questions

Question	Short Answer
Why separate income and expense tables?	They have different fields and behavior but can be combined for UI.
How are they combined?	FirebaseTransactionRepository combines income and expense flows.
How is month filtering done?	By comparing transaction dateMillis with selected month start/end.
How are custom categories stored?	In profile categorySettingsJson.
How is exchange rate handled?	Converted amount is stored in amountCents; original values and rate are also stored.

K. Dashboard Questions

Question	Short Answer
What does Dashboard show?	Balance, monthly income, monthly expense, net savings, goals, and recent transactions.
Where does data come from?	From ViewModel state built from Room-backed repository flows.
What date field is used?	dateMillis, the user-selected event date.
Why show yearly savings?	It gives a longer-term financial summary beyond current month.
Does Dashboard edit transactions?	It mainly displays recent transactions; edit/delete belongs to History.

L. Goals Questions

Question	Short Answer
How are goals represented?	SavingsGoal model and SavingsGoalEntity Room table.
What is isPrimary?	Marks a goal for the primary goals section.
What is iconKey?	Stores the selected goal icon name.
How is added savings saved?	It updates goal saved amount and creates linked Goal expense.
What validation prevents over-saving?	Added amount cannot exceed remaining target.

M. Analytics Questions

Question	Short Answer
Are analytics live?	Yes, based on transaction records.
Where are calculations done?	AnalyticsViewModel.
What does the donut chart show?	Expense distribution by category.
What is committed spending?	Expenses marked committed or in committed default categories like Rent/Gym/Subscriptions.
What does monthly overview show?	Income and expense bars for the last 5 months relative to selected month.

N. Profile Questions

Question	Short Answer
What profile data is stored?	Name, email, currency, image URI, category settings, theme mode,

Question	Short Answer
	timestamps.
Is profile image uploaded to Firebase?	No, local URI only. Firebase Storage is not implemented.
How is theme changed?	Profile Appearance row updates themeMode; MainActivity applies theme.
How is currency changed?	Profile updates default currency through repository.
How is logout handled?	Firebase Auth sign-out through AuthRepository.

O. UI/UX Questions

Question	Short Answer
What design system is used?	Material Design 3 with Spendly colors and shared components.
How is dark mode supported?	Light/dark color schemes and profile theme mode.
How are FABs made consistent?	Shared FAB/menu component and scaffold-level placement.
How is month picker consistency handled?	Shared SpendlyMonthPicker.
Why use reusable components?	Consistent UI and less duplicated code.

P. Testing And Future Improvement Questions

Question	Short Answer
Are unit tests present?	Only placeholder/default tests are visible; meaningful tests should be added.
What should be tested first?	Auth, add transaction, sync, goal savings, analytics calculations.
What is a major limitation?	Firestore security rules need updating for current schema.
What sync improvement is needed?	Delete tombstones and stronger conflict resolution.
What feature could be added next?	Budgets, reminders, exports, richer analytics, cloud profile image storage.

19. Important Project Notes

19.1 Strengths

- Real MVVM architecture.
- Compose Material 3 UI.
- Firebase Authentication integration.
- Firestore per-user data structure.
- Room local database with migrations.
- Hilt dependency injection.
- WorkManager background sync.
- Local-first write flow.
- Separate domains for team members.
- Shared UI components for consistency.
- Analytics calculated from actual data.
- Goal savings integrated with expense records.
- Theme selector with System/Light/Dark modes.

19.2 Current Limitations

Limitation	Explanation
Firestore rules mismatch	Current firestore.rules appears to target old transactions schema, not current
Delete sync	income/expenses/goals/profile paths
Profile image cloud storage	Offline-safe delete tombstones are not found
Automated tests	Firestore Storage upload is not implemented
Remote cascading delete	Meaningful unit/instrumentation tests are limited or not found
Full conflict resolution	Deleting account/goal may not fully clean nested subcollections
API reliability	Newest timestamp merge exists, but complex multi-device conflicts are limited
	Currency/crypto API may fail, but manual rate fallback exists

19.3 Known Bugs Or Risk Areas Visible From Code

- Existing Firestore rules may block or fail to protect the currently used paths unless updated in Firebase Console.
- Room exportSchema=false reduces migration auditability.
- Firestore subcollection deletion is not automatically handled by deleting parent user document.
- Profile image URI may not work across devices because it is local to the device.
- External exchange-rate services are optional and may not always return data.

- If local delete happens offline, a robust remote tombstone pattern is not found.

19.4 What To Prepare Before Demo

1. Use a test Firebase account.
2. Add at least two income records.
3. Add at least five expense records across categories.
4. Add one committed expense such as Rent or Subscription.
5. Add one discretionary expense such as Food.
6. Add at least two goals.
7. Mark one goal as primary.
8. Add savings to a goal.
9. Ensure Analytics has data for the selected month.
10. Test Profile theme switch.
11. Test History month picker and filters.
12. Confirm Firestore database contains user data under the correct UID.

19.5 Screens To Test Before Demo

Screen	Test
Splash	Routes correctly
Login	Sign in and forgot password message
Create Account	Validation and profile creation
Dashboard	Values update after adding records
Add Income	Save default currency and USD/manual rate
Add Expense	Category, payment, type, date, save
History	Filter, edit, delete
Goals	Add, edit, add savings, prevent over-save
Analytics	Month picker, chart, split, overview
Profile	Edit name/photo, currency, theme, password, logout

20. Future Improvements

20.1 Better Offline-First Sync

Add tombstone records for deletes so offline deletions can sync later. Add stronger conflict resolution for simultaneous edits on multiple devices.

20.2 Better Firestore Security Rules

Update `firestore.rules` to match current paths:

- `users/{uid}/profile/main`
- `users/{uid}/income/{incomeId}`
- `users/{uid}/expenses/{expenseId}`

- `users/{uid}/goals/{goalId}`

Also validate important fields such as `userId`, `amountCents`, `dateMillis`, and `updatedAtMillis`.

20.3 Unit And Integration Tests

Recommended tests:

- ViewModel validation tests.
- Analytics calculation tests.
- Repository mapping tests.
- Room migration tests.
- Goal savings tests.
- Transaction delete/update tests.

20.4 Better Error Handling

Add consistent error banners/snackbars across all screens for:

- Network failure.
- Firestore permission denied.
- Invalid exchange rate.
- Sync failure.
- Password reauthentication failure.

20.5 More Analytics Charts

Future analytics:

- Weekly trend.
- Budget vs actual.
- Top merchants.
- Highest expense category.
- Savings goal projection.
- Exportable reports.

20.6 Budgeting Feature

Add category budgets:

- Monthly budget per category.
- Warning when near limit.
- Budget usage chart.
- Budget carry-over.

20.7 Notification Reminders

Use `WorkManager` or `AlarmManager` for reminders:

- Daily expense reminder.

- Bill due reminder.
- Goal contribution reminder.

20.8 Cloud Profile Images

Add Firebase Storage:

- Upload profile image.
- Store download URL in profile.
- Sync profile picture across devices.

20.9 Performance Improvements

- Add indexes if Firestore queries become more complex.
- Add pagination for long transaction histories.
- Keep heavy calculations in ViewModels or repository layer.
- Add derived analytics caching if data volume grows.

20.10 Accessibility Improvements

- Add content descriptions for icons.
- Improve TalkBack labels.
- Test color contrast in dark mode.
- Support larger font sizes.

21. Final Quick Revision Sheet

21.1 Ten Most Important Files

#	File	Why Important
1	MainActivity.kt	Starts app and applies theme
2	SpendlyApplication.kt	Hilt and WorkManager setup
3	ui/FinanceTrackerApp.kt	Navigation, scaffold, bottom nav, FAB
4	ui/viewmodel/FinanceViewModel.kt	Global session/dashboard/profile state
5	data/local/db/SpendlyDatabase.kt	Room DB and migrations
6	di/AppModule.kt	Provides Firebase, Room, DAOs, WorkManager
7	data/repository/FirebaseAuthRepository.kt	Firebase Authentication

#	File	Why Important
8	data/remote/IncomeRepositoryImpl.kt	Income Room/Firestore logic
9	data/remote/ExpenseRepositoryImpl.kt	Expense Room/Firestore logic
10	worker/SpendlySyncWorker.kt	Background sync

21.2 Ten Most Important Flows

1. App startup and splash routing.
2. Firebase login and session observation.
3. Create account and profile creation.
4. Add income local-first save.
5. Add expense local-first save.
6. History filtering and grouping by `dateMillis`.
7. Dashboard calculations from transaction flows.
8. Goal creation and progress calculation.
9. Goal savings linked expense creation.
10. WorkManager sync of unsynced Room rows.

21.3 Ten Most Important Viva Answers

1. “The app uses MVVM with repositories to separate UI from data access.”
2. “Room is used as the local source for UI state and offline cache.”
3. “Firestore is used for per-user cloud sync under `users/{uid}`.”
4. “Firebase UID is the owner key for all user records.”
5. “Money is stored as cents in Long fields to avoid floating-point errors.”
6. “Transactions use `dateMillis` for financial reporting and `createdAtMillis` for audit.”
7. “ViewModels expose state to Compose and keep calculations out of UI.”
8. “Hilt injects Firebase, Room, DAOs, repositories, ViewModels, and the worker.”
9. “WorkManager retries sync when network is available.”
10. “Current Firestore rules should be updated because the rules file appears older than the implemented schema.”

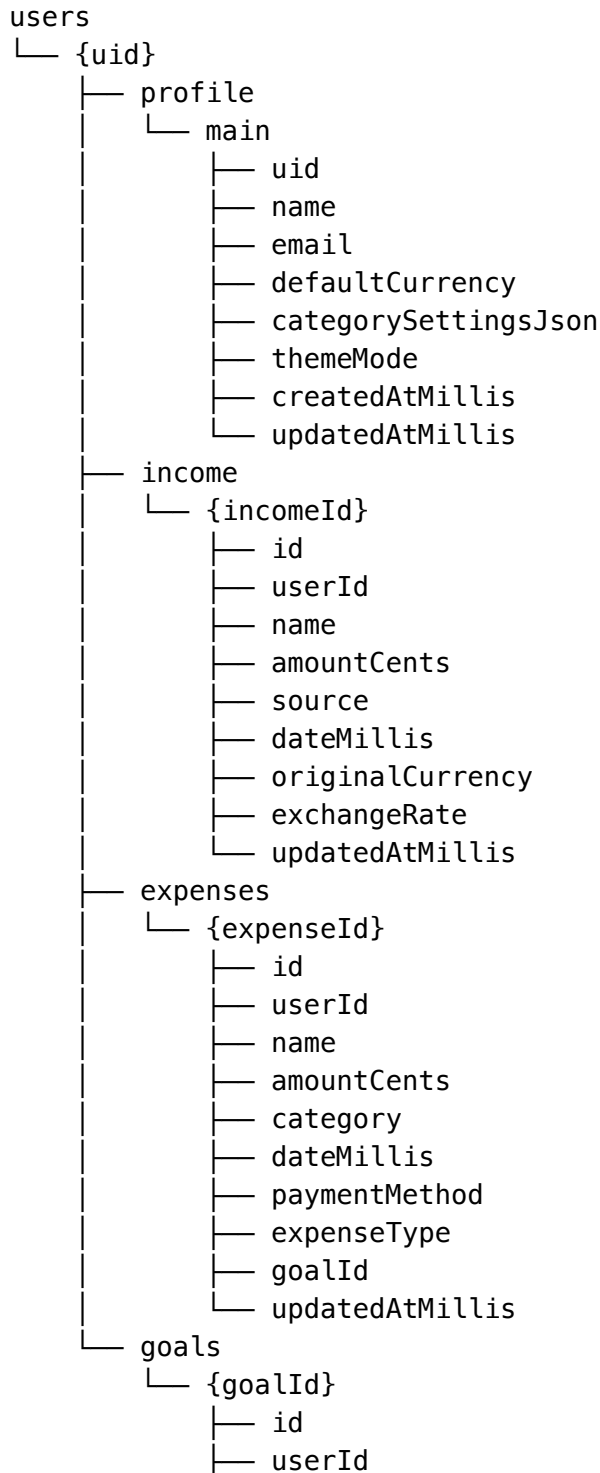
21.4 Ten Things To Test Before Demo

1. Login with existing Firebase account.
2. Create account with default currency.
3. Add income and confirm Dashboard updates.
4. Add expense and confirm Dashboard updates.
5. Edit and delete transaction from History.
6. Change History month.
7. Create a goal and add savings.
8. Confirm goal saving appears as expense.
9. Open Analytics and verify charts use live data.
10. Change Profile theme to Dark/Light/System.

21.5 Five Strongest Technical Points To Mention

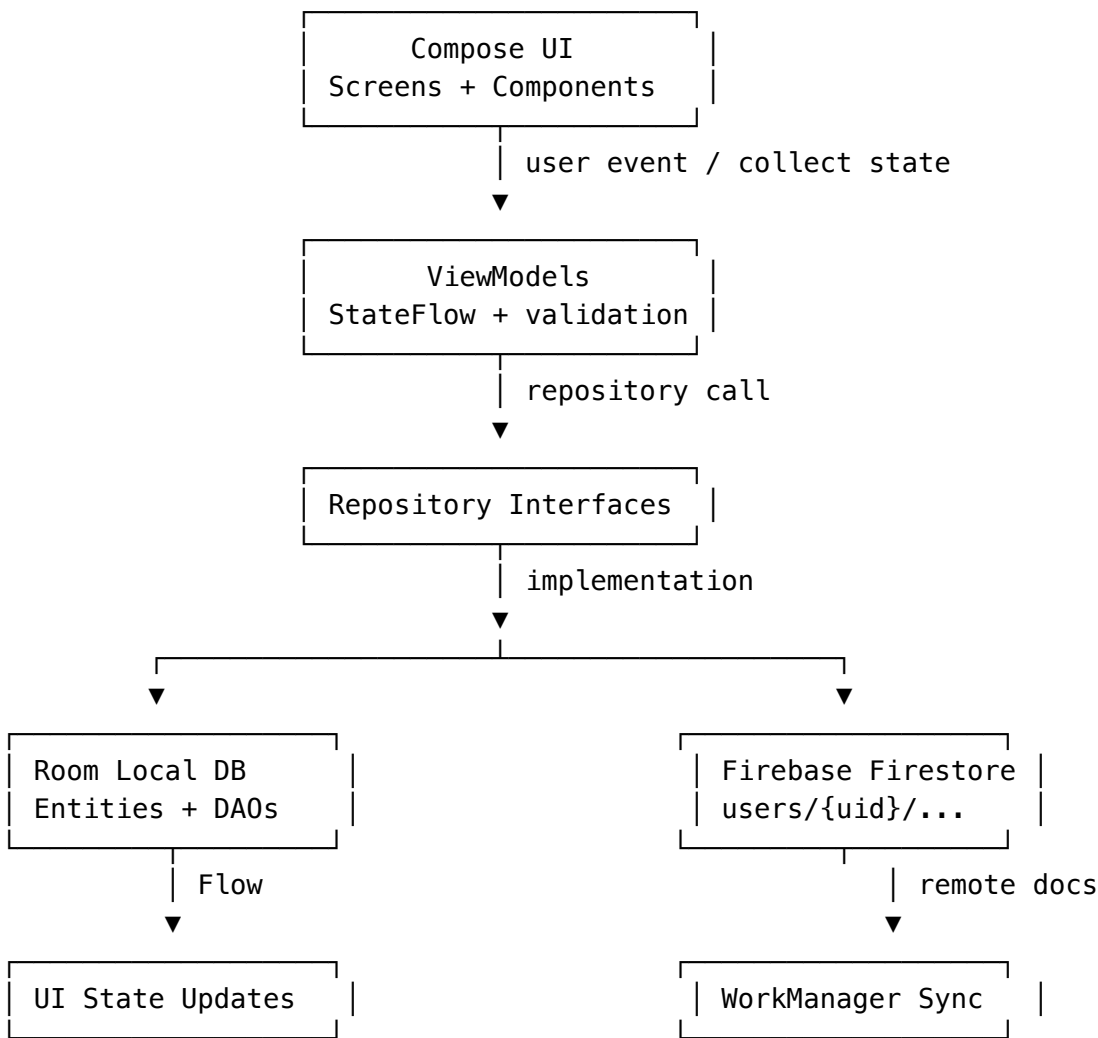
1. Local-first Room + Firestore sync architecture.
 2. Clean MVVM separation with repository interfaces.
 3. Hilt dependency injection across app, repositories, database, and worker.
 4. WorkManager background sync with network constraints.
 5. Data-driven analytics and dashboard using actual persisted transactions.
-

Appendix A. Complete Firestore Structure To Explain



- title
- targetCents
- savedCents
- dueDateMillis
- isPrimary
- iconKey
- updatedAtMillis

Appendix B. Architecture Diagram



Appendix C. Not Found In Current Codebase

These items were requested or are common in production apps, but are not found or not fully implemented in the current repository:

Item	Status
Firestore security rules matching current income/expenses/goals/profile schema	Not found in current codebase

Item	Status
Firebase Storage upload for profile image	Not found in current codebase
Room TypeConverter file	Not found in current codebase
Meaningful unit tests for repositories/ViewModels/calculations	Not found in current codebase
Full delete tombstone sync	Not found in current codebase
Automatic cascading delete of all Firestore user subcollections on account delete	Not found in current codebase
Exported Room schema JSON files	Not found in current codebase